1. INTRODUCTION

1.1 Introduction

The e-Certificate Management System is a comprehensive web-based full-stack application designed to digitize and streamline the processes of certificate request, approval, generation, and verification within academic institutions. By replacing traditional manual methods, it addresses inefficiencies in paper-based certificate handling, offering a secure, efficient, and user-friendly digital alternative. The system leverages a modern technology stack, including React for a responsive and interactive frontend, Node.js and Express for robust backend API development, and MySQL as the relational database to ensure reliable data storage and retrieval. It incorporates role-based access control, allowing students, administrators, and public verifiers to interact with tailored modules based on their permissions, thereby enhancing security and usability.

Students benefit from a seamless experience: they can register and log in using email-based OTP verification, submit detailed certificate requests (such as Bonafide, Conduct, or Scholarship certificates) through intuitive forms, track application statuses in real-time via personalized dashboards, and download approved certificates as downloadable PDF files. Administrators, on the other hand, access a dedicated dashboard to review pending requests, approve or reject applications with optional remarks, select from customizable HTML/CSS templates, automatically generate QR code-embedded certificates, and monitor comprehensive verification logs for auditing purposes. Public verifiers, including employers or institutions, can independently validate certificates by scanning the embedded QR code or entering the unique reference number on a public verification page, receiving instant confirmation of authenticity.

Each generated certificate includes a unique reference identifier and an embedded QR code that links directly to a secure backend verification endpoint. Upon scanning, the system queries the database to retrieve and display certificate details, ensuring tamper-proof integrity and preventing forgery. The application adheres to industry-standard security practices, including bcrypt-based password hashing, encrypted OTP storage with expiration, and robust protections against SQL injection through parameterized queries and cross-site scripting via input sanitization. This approach not only reduces administrative overhead and processing times but also fosters trust through verifiable, auditable digital records, making it an essential tool for modern educational institutions.

1.2 Problem Statement

Educational institutions face several challenges in managing and verifying academic certificates:

- Manual preparation and printing of certificates increases processing time and administrative workload.
- Physical certificates are vulnerable to loss, damage, or unauthorized alterations.
- There is no fast, standardized way for third parties to verify the authenticity of certificates without directly contacting the institution.
- Maintaining paper-based records makes retrieval and auditing difficult as the volume of certificates grows over time.
- Manual verification methods are inefficient and may still fail to detect sophisticated tampering.

Problem Statement:

Design and develop a secure, scalable, and user-friendly e-Certificate Management System that automates certificate request, approval, generation, and verification processes, while ensuring authenticity, integrity, and easy accessibility of academic certificates.

1.3 Objectives of the Project

1.3.1 Primary Objectives

- To develop a full-stack web application for end-to-end management of academic certificates in a college or university.
- To reduce the time required for certificate issuance from days/weeks to a few minutes through automation.
- To provide students with online access to their approved certificates in downloadable PDF form.
- To enable external verifiers to validate certificates instantly using QR codes or reference numbers without contacting the institution manually.

- To maintain a centralized, digital repository of all certificates and verification logs for auditing and reporting.

1.3.2 Technical Objectives

- To implement secure user authentication and authorization with role-based access control for students and administrators.
- To design a normalized relational database schema that efficiently stores users, certificates, templates, OTP records, and verification logs.
- To build RESTful API endpoints for all core operations including registration, login, certificate requests, approvals, generation, and verification.
- To integrate PDF generation and QR code libraries for creating professional, machine-verifiable certificates.
- To implement best practices in web security, including password hashing with bcrypt, encrypted OTP storage, and protection against common web vulnerabilities.

2. LITERATURE SURVEY

2.1 Introduction to Digital Certificate Systems

Digital certificate management systems have evolved as a response to the limitations of manual, paper-based methods used in academic institutions and professional certification bodies. Such systems focus on digitizing the creation, distribution, and verification of credentials, often leveraging QR codes, cryptographic techniques, or blockchain to improve trust and traceability. This section reviews existing solutions, both commercial and academic, to identify gaps that the proposed e-Certificate Management System aims to address.

2.2 Existing Systems and Approaches

Commercial platforms such as Credly and Accredible provide cloud-based solutions for issuing digital credentials, mainly targeting professional training and online courses. These platforms often support rich metadata, badge-style certificates, and integrations with Learning Management Systems (LMS), but they may involve subscription costs and vendor lock-in for educational institutions.

Research systems like the *Student Certificate Verification System* use web applications combined with QR codes and image analysis techniques to detect certificate tampering and verify authenticity. Some works explore blockchain-based approaches where certificate hashes are stored on a distributed ledger to provide tamper-evident verification. While these systems demonstrate strong security, they can be complex and costly to deploy for smaller institutions.

Manual or semi-digital approaches continue to dominate in many universities, where certificates are generated in word processors, printed, signed, and later scanned or emailed. Verification is usually done through email or phone, which is time-consuming and not standardized. These limitations highlight the need for an institution-controlled, open-source-friendly system that uses standard web technologies and can be deployed on local infrastructure.

2.3 Market Analysis and Gap Identification

There is a clear market demand for solutions that combine the security and convenience of modern digital platforms with the flexibility and cost-effectiveness required by academic institutions, especially in developing regions. Many colleges need internal, on-premises systems that do not rely on third-party cloud vendors due to privacy, compliance, or budget constraints.

Existing commercial systems may offer rich features but are not always customizable to specific institutional formats, certificate templates, or local workflows. Research prototypes often focus on specific aspects such as blockchain or image forensics rather than providing a complete, production-ready implementation. The proposed e-Certificate Management System aims to fill this gap by offering:

- A full-stack, institution-hosted web application using widely adopted technologies.
- A complete workflow from request to verification integrated in one system.
- QR code–based verification with a familiar browser-based interface.
- Open extensibility for future integration with blockchain, digital signatures, or ERP systems.

3. THEORETICAL BACKGROUND

3.1 Software Engineering and SDLC Concepts

The project follows standard Software Development Life Cycle (SDLC) principles, including requirement analysis, design, implementation, testing, deployment, and maintenance. An iterative or incremental approach is appropriate for web applications, allowing the team to build a minimal viable product first and gradually add features such as templates, dashboards, and analytics.

A feasibility study prior to development evaluates economic, technical, operational, and schedule-related aspects to ensure that the project is viable and aligned with institutional needs. This structured approach reduces risks and improves quality and maintainability.

3.2 Web Application Architecture and MVC

The application follows a layered architecture with a clear separation of concerns between frontend, backend, and database layers. The Model–View–Controller (MVC) pattern is applied conceptually: the database entities act as models, React components act as views, and Express routes and controllers act as controllers handling business logic.

In this pattern, the view collects user input and sends requests to the backend controller via HTTP; the controller interacts with the model (database) and returns responses that update the UI. This separation enhances testability, reusability, and maintainability of the code base.

3.3 Security and Authentication Concepts

Secure user authentication is critical in any system handling personal and academic data. The project uses industry-standard techniques:

- **Password Hashing:** Passwords are hashed using bcrypt, which incorporates salting and a configurable cost factor to resist brute-force attacks.
- **OTP-Based Verification:** One-Time Passwords are generated for email verification and password reset; OTP values are encrypted before being stored in the database to limit exposure even if the database is compromised.
- **Web Security Basics:** Parameterized queries prevent SQL injection, while input sanitization and output encoding reduce cross-site scripting (XSS) risks. CSRF is mitigated through session validation and secure cookies.

3.4 Database Design and Normalization

A relational database such as MySQL is used to store users, certificate requests, templates, OTP records, and verification logs. Database design follows normalization principles up to Third Normal Form (3NF) to minimize redundancy and ensure consistency.

The core entities include:

- **Users:** Stores login credentials and profile details for students and administrators.
- **Certificates:** Stores request details, status, reference numbers, file paths, and links to templates.
- **OTP Records:** Stores encrypted OTP values and expiry times for security flows.
- **QR Verification Logs:** Stores records of QR-based or manual verifications, including IP address and user agent for auditing.

3.5 Certificate Generation, QR Codes, and Verification

Certificate documents are generated in PDF format using server-side libraries that render HTML/CSS templates and embed images such as logos, signatures, and QR codes. Each certificate is assigned a unique reference number and a verification URL in the form /verify/{referenceNumber}.

QR codes encode this verification URL in a machine-readable 2D barcode that can be scanned using standard smartphone cameras. When scanned, the browser opens the verification page, where the backend looks up the reference number in the database and returns a result indicating whether the certificate is valid, revoked, or not found. This mechanism provides fast, reliable authenticity checks without exposing internal implementation details.

4. SYSTEM REQUIREMENTS

4.1 Hardware Requirements

Minimum System Configuration

- Processor: Intel Core i5/i7 or equivalent
- RAM: 8–16 GB (for better performance under concurrent load)
- Storage: 256 GB or higher SSD for faster I/O and backups
- Network: Gigabit Ethernet and reliable internet for email and updates

4.2 Software Requirements

- **Operating System:** Windows 10/11, Linux (Ubuntu 18.04+), or macOS.
- **Backend Runtime:** Node.js (v14 or later) and npm.
- **Frontend Framework:** React.js with React Router.
- **Database:** MySQL Server 5.7 or 8.0.
- **Tools:** Visual Studio Code (IDE), Git (version control), Postman or Thunder Client for API testing.

4.3 Functional Requirements

- **User Registration and Authentication:**
    - Students can register with email, enrollment number, and department.
    - System sends OTP for email verification.
    - Users can log in using email and password.
- **Certificate Request and Approval:**
    - Students can submit requests specifying certificate type and academic details.
    - Administrators can view pending requests and approve or reject them with remarks.
- **Certificate Generation and Download:**

    - On approval, the system generates a PDF certificate with a unique reference number and an embedded QR code.
    - Students can view and download approved certificates.
- **Public Verification:**
    - Anyone can verify a certificate by scanning the QR code or entering a reference number on the verification page.
- **Notifications:**
    - OTP, password reset, and certificate status notifications are sent via email.

4.4 Non-Functional Requirements

- **Performance:**
    - Average API response time below 500 ms under normal load.
    - Certificate generation time within a few seconds for single requests.
- **Security:**
    - All passwords hashed using bcrypt; no plaintext passwords in the database.
    - OTP values encrypted; no sensitive information logged.
- **Reliability and Availability:**
    - High availability on institutional LAN during working hours.
    - Automated backups of database and certificate files.
- **Scalability:**
    - Ability to handle thousands of certificates and hundreds of concurrent users, with scope to scale by upgrading hardware or distributing components.
- **Usability and Maintainability:**
    - Intuitive web interface requiring minimal training.
    - Clean, modular code structure following best practices in full-stack development.

5. FEASIBILITY STUDY

5.1 Economic Feasibility

Economic feasibility examines whether the benefits of the system justify the costs of development and maintenance. For this project, the software stack is entirely based on open-source technologies (Node.js, React, MySQL), eliminating licensing costs. Hardware requirements are modest and can be satisfied using existing institutional servers or lab machines in many cases.

The primary benefits include reduced administrative workload, faster processing times, lower paper and printing usage, and improved reputation through reliable verification. Compared to commercial certificate platforms that typically charge subscription or per-certificate fees, an in-house system significantly reduces recurring costs. Therefore, the project is considered economically feasible.

5.2 Technical Feasibility

Technical feasibility assesses whether the project can be implemented with the available technology stack, skills, and infrastructure. The technologies chosen—React, Node.js, Express, and MySQL—are widely used in industry and supported by extensive documentation and community resources.

Academic reports and training materials demonstrate successful use of similar stacks for full-stack web applications, confirming the suitability of this approach. The development team possesses core web development skills, and institutional infrastructure can support local hosting. As a result, there are no significant technical barriers, and the project is technically feasible.

5.3 Social and Operational Feasibility

Social feasibility evaluates how well the proposed system will be accepted by stakeholders and how easily it can be integrated into day-to-day processes. Students benefit from faster, online access to certificates, which reduces the need for repeated visits to the office. Administrative staff benefit from reduced manual data entry and filing, allowing them to focus on more critical academic tasks. The user interface is designed to be simple and intuitive, requiring minimal training for both students and staff. Because verification is web-based and does not require special software, external verifiers can quickly adapt to the system. Thus, the solution is socially and operationally feasible.

6. SYSTEM ARCHITECTURE & DESIGN

6.1 Architectural Layers

The e-Certificate Management System follows a layered architecture with clear separation between presentation, application, data access, and infrastructure concerns.

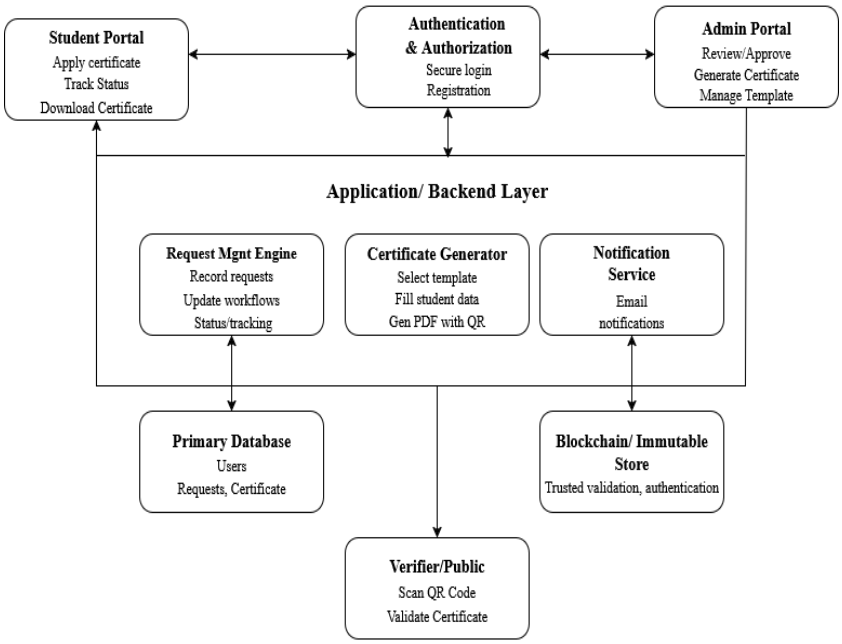

Figure 6.1: System Architecture

- **Presentation Layer (Frontend):**
  Implemented using React, this layer handles all user interface components, routing, and user interactions in the browser.

- **Application Layer (Backend):**
  Implemented using Node.js and Express, this layer exposes RESTful APIs, performs business logic, and enforces validation and security checks.

- **Data Access Layer (Database):**
  Uses MySQL with a connection pool to execute parameterized queries and manage all persistent data.

- **Infrastructure Layer:**
  Includes the file system for storing PDFs and QR code images, SMTP server for sending emails, and environment configuration for secure credentials.

6.2 UML Diagrams

To better understand system behaviour and interactions, UML diagrams are used.

6.2.1 Use Case Diagram

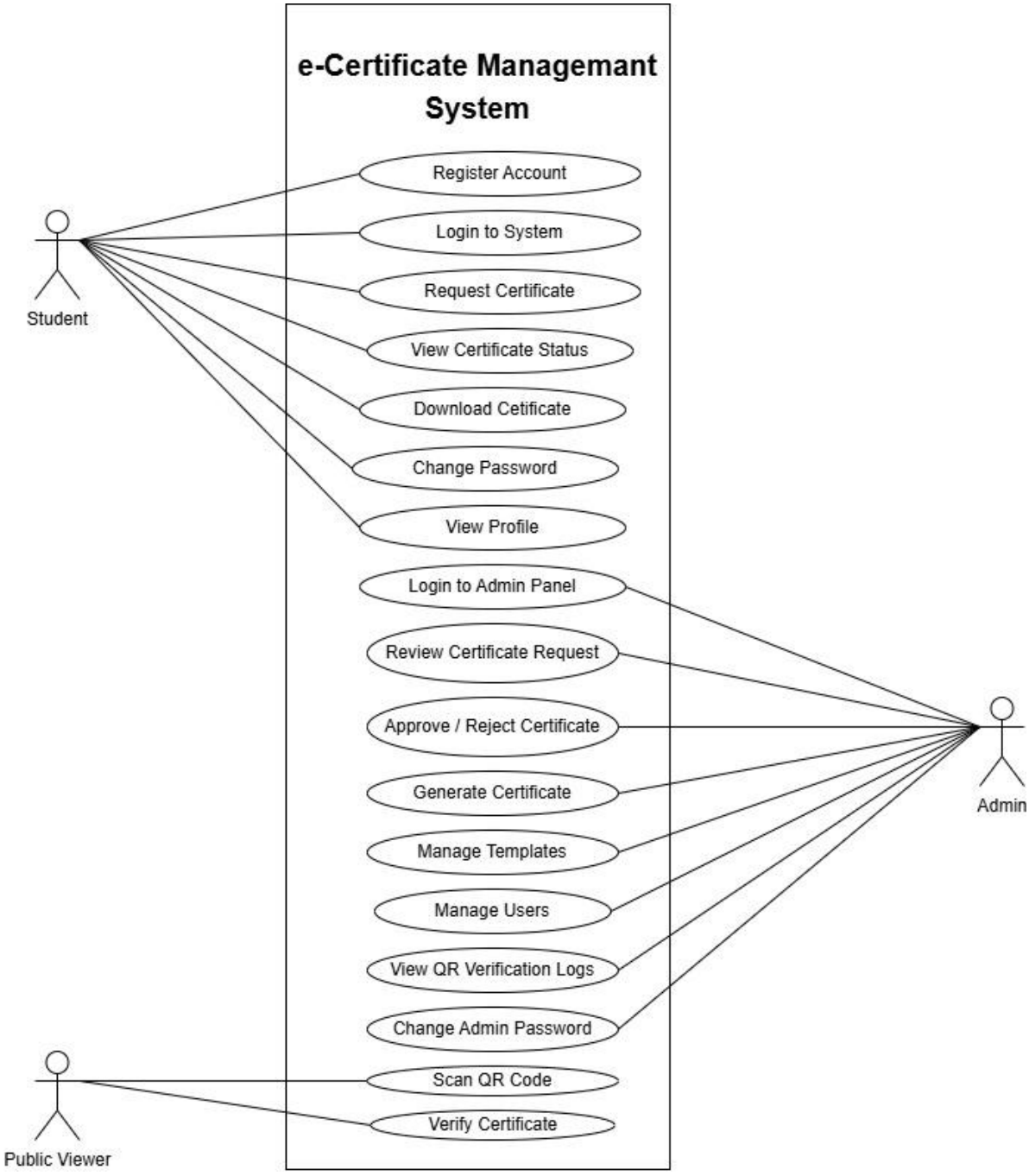

Figure 6.2.1: Use Case Diagram

This diagram identifies major use cases such as “Register”, “Request Certificate”, “Approve Request”, “Generate Certificate”, and “Verify Certificate”.

6.2.2 Class Diagram

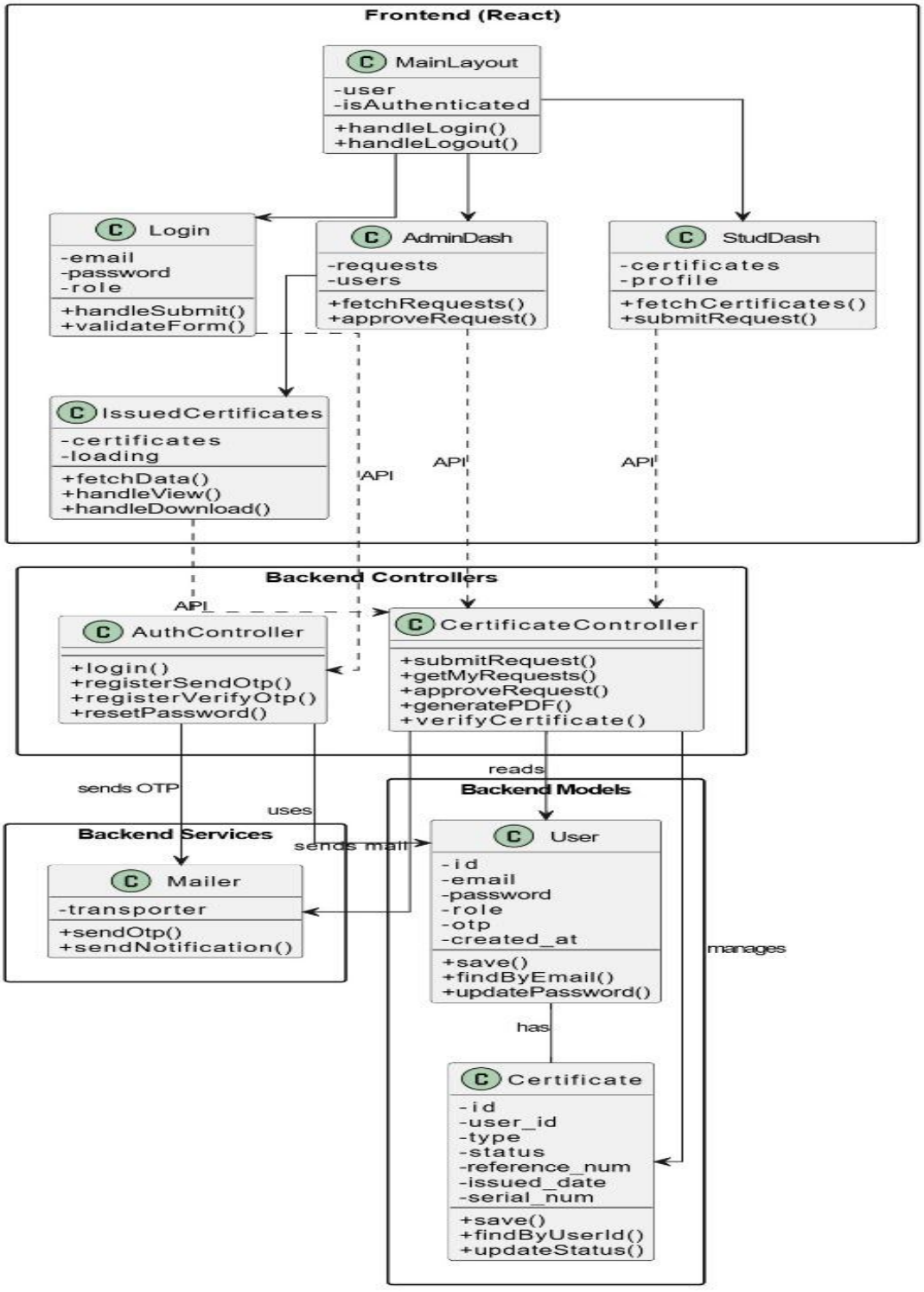

Figure 6.2.2: Class Diagram

The class diagram shows core classes, their attributes, and relationships, corresponding to database tables and business objects.

6.2.3 Sequence Diagram

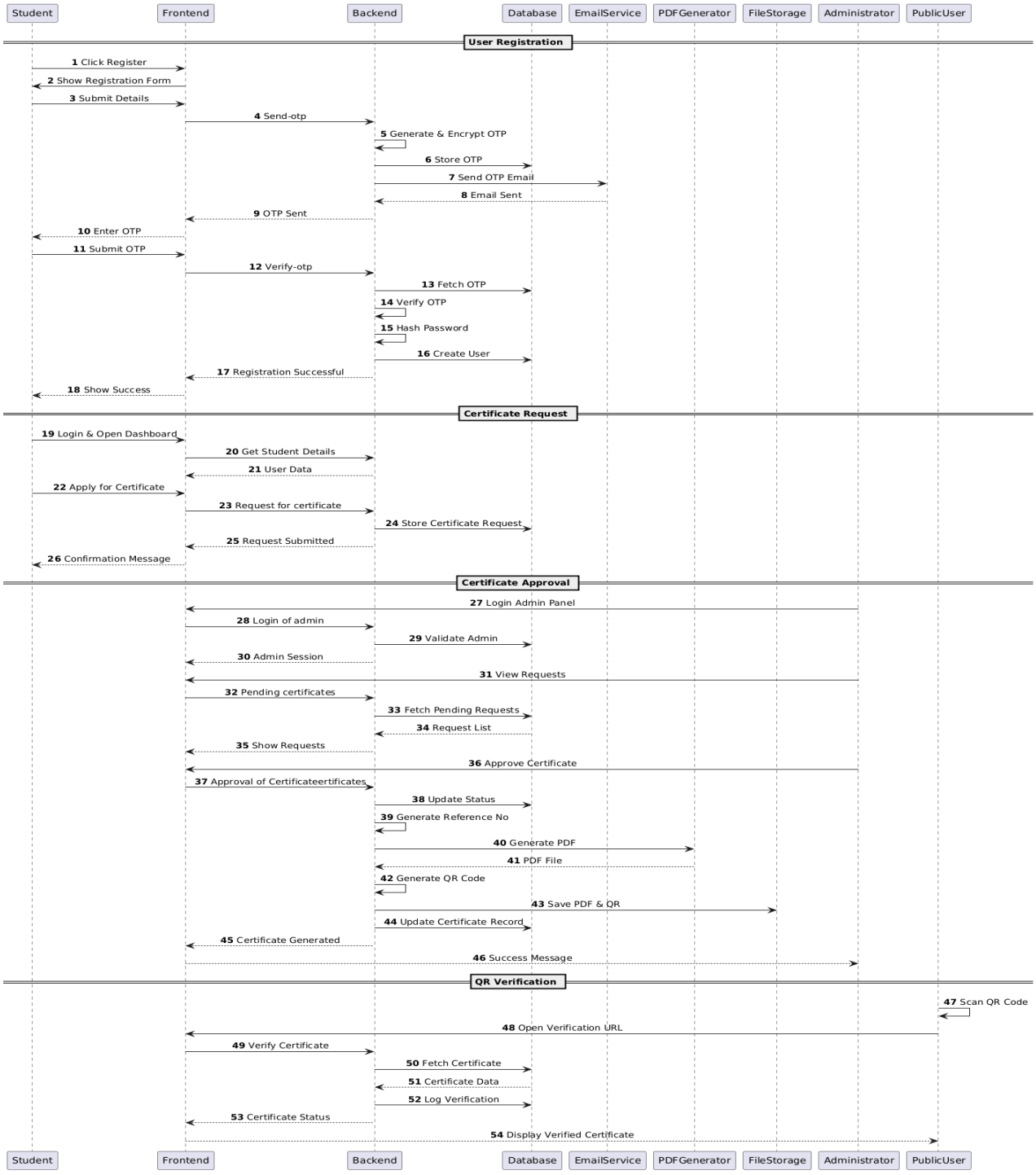

Figure 6.2.3: Sequence Diagram

This sequence diagram illustrates the interaction between Student, Admin, Frontend, Backend, Database, and File System during the approval and generation of a certificate.

6.2.4 Activity Diagram

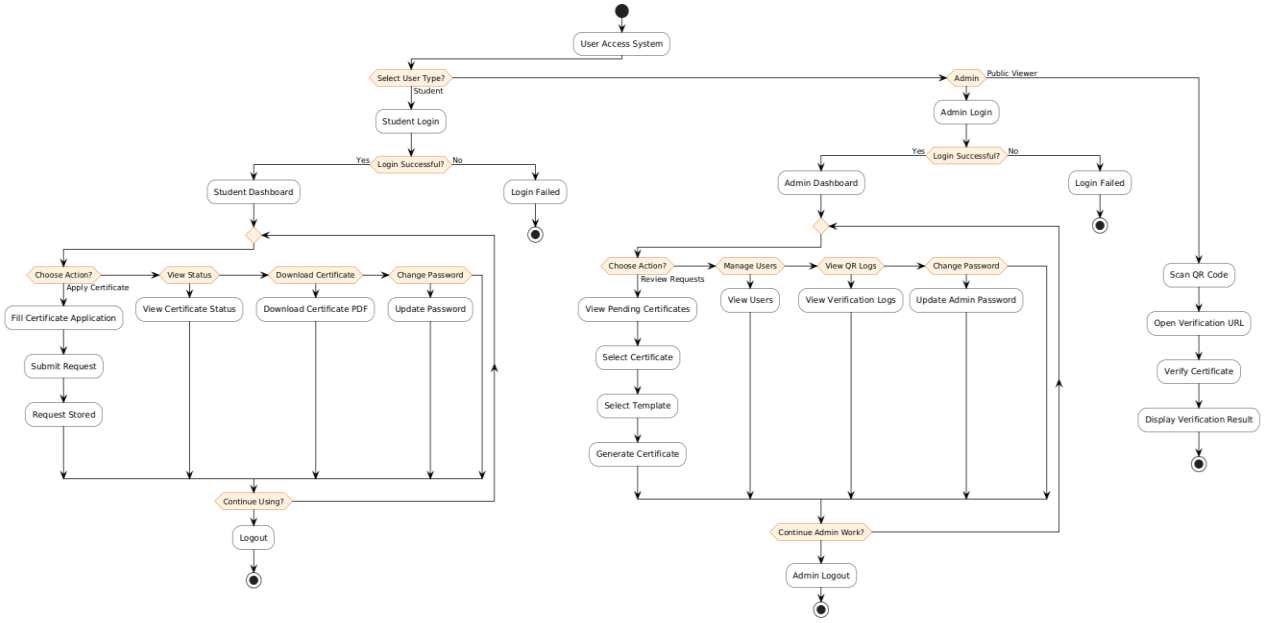

Figure 6.2.4: Activity Diagram

This activity diagram models the complete user access workflow for students, admins, and public viewers, from login to logout, including QR-based verification. It clearly shows each role's possible actions like applying for certificates, managing users, viewing logs, and verifying certificates.

6.3 Database Schema

The database schema consists of multiple related tables:

- **users (id, username, email, password, role, enrollment_number, department, created_at, updated_at)**

```
mysql> desc users;
+--------------+--------------+------+-----+-------------------+-------------------+
| Field        | Type         | Null | Key | Default           | Extra             |
+--------------+--------------+------+-----+-------------------+-------------------+
| id           | int          | NO   | PRI | NULL              | auto_increment    |
| full_name    | varchar(100) | YES  |     | NULL              |                   |
| username     | varchar(100) | NO   | MUL | NULL              |                   |
| gender       | varchar(10)  | YES  |     | NULL              |                   |
| password     | varchar(128) | NO   |     | NULL              |                   |
| roll_number  | varchar(20)  | YES  |     | NULL              |                   |
| phone_number | varchar(15)  | YES  |     | NULL              |                   |
| course       | varchar(100) | YES  |     | NULL              |                   |
| branch       | varchar(100) | YES  |     | NULL              |                   |
| email_id     | varchar(255) | NO   | MUL | NULL              |                   |
| role         | int          | YES  |     | 0                 |                   |
| created_date | datetime     | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+--------------+--------------+------+-----+-------------------+-------------------+
12 rows in set (0.11 sec)
```

Figure 6.3.1: Describe Users

mysql> select * from users;

id	full_name	username	gender	password	roll_number	phone_number	course	branch	email_id	role	created_date
1	ADMIN UCET MGU	admin_ucet	NULL	$2b$10$ToHK9gbHbAo3U34NQ0lgeuct7p3HNY.l9Qzr5lUUYIpOhzIHBSbdu	NULL	NULL	NULL	NULL	mguecertificate@gmail.com	1	2025-10-22 16:29:54
2	Devarakonda Ashresh Kumar	ashresh_devarakonda	Male	$2b$10$rrzuzrFhHsGylsiH05TX6Oo8KULHMqsnGLDk7/6nz73hn6GSzFpBq	451122733008	9391448946	B.Tech	CSE	dashreshkumar@gmail.com	0	2025-10-22 16:32:06
3	Mannem Pooja	pooja_mannem	Female	$2b$10$UO9eyuGbZYnuHaFUr.Gxz.kKQLEq67tnT9OvHkKytmcrfUI1sHZvi	451122733015	6301966181	B.Tech	CSE	poojamannem@gmail.com	0	2025-10-25 20:26:12
4	Vallamalla Harshitha	honey	NULL	$2b$10$3Mnj0782ZSRSalBHQwOW4u.1MvFWuN5gCrXG9nYz/3qGd85HbZequ	451124733008	8985071974	B.Tech	CSE	vallamallaharshitha1974@gmail.com	0	2025-10-25 20:41:41
5	Abbula Ishwarya	ishu_abbula	Female	$2b$10$m0zk62DFJ8uKGYDuD42G8OzzPok7E/.DQg/5qfExdejHXZN3tIjza	451123733008	9177980751	B.Tech	CSE	aishwaryaabbula@gmail.com	0	2025-10-25 21:07:33
6	Chata Vinay Kumar	chatavinaykumar@gmail.com	Male	$2b$10$wXpDelo0Ob/sy.Tlh32LSuTWO/0xFtlhygViVLJ6OBLkow0LkqYUW	451122733664	8978520357	B.Tech	CSE	chatavinaykumar@gmail.com	0	2025-12-15 22:17:28
7	Maddikuntla Likhitha	likitha	Female	$2b$10$hIZlwKmKB076aIQYxSlqPe/h.tZEg1ejJoNRrbY0D.N4xuiHezNUa	451122733022	8179546106	B.Tech	CSE	lkhithamaddikuntla@gmail.com	0	2025-12-16 12:12:00
8	Neelima Dogiparthi	neelima	Female	$2b$10$lzD8ebwaQ.Tz2BYDFrpDhOmdUisZhF1aPIyT/S4dM.yxYeGG6ysEu	451122733004	7794038394	B.Tech	CSE	dbhagyalaxmi2104@gmail.com	0	2025-12-16 13:59:14

8 rows in set (0.01 sec)

Figure 6.3.2: Users Data

- **certificates (id, reference_num, user_id, certificate_type, course_name, department, academic_year, status, template_id, pdf_path, qr_code_path, created_at, approved_at, approved_by)**

mysql> desc certificates;

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
serial_num	varchar(50)	NO		NULL	
reference_num	varchar(50)	NO		NULL	
certificate_type	varchar(50)	NO		NULL	
full_name	varchar(100)	NO		NULL	
parent_name	varchar(100)	YES		NULL	
roll_number	varchar(20)	NO		NULL	
course	varchar(50)	YES		NULL	
branch	varchar(50)	YES		NULL	
duration	varchar(15)	YES		NULL	
qr_code	varchar(255)	YES		NULL	
pdf_path	varchar(255)	YES		NULL	
requested_date	datetime	NO		CURRENT_TIMESTAMP	DEFAULT_GENERATED
issued_date	datetime	YES		NULL	
status	varchar(20)	YES		pending	
remark	varchar(255)	YES		NULL	
created_at	timestamp	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED
updated_at	timestamp	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED on update CURRENT_TIMESTAMP
parent_prefix	varchar(8)	NO		Mr.	
present_year	varchar(50)	YES		NULL	

20 rows in set (0.01 sec)

Figure 6.3.3: Describe certificates

mysql> select * from certificates;

id	serial_num	reference_num	certificate_type	full_name	parent_name	roll_number	course	branch	duration
1	CERT/2025/37320	45112025925863	Scholarship Bonafide Certificate	Devarakonda Ashresh Kumar	Narendar	451122733008	B.Tech	CSE	2022-2026
2	CERT/2025/17269	45112025387478	Bonafide Certificate	Mannem Pooja	eshwari	451122733015	B.Tech	CSE	2022-2026
3	CERT/2025/59953	45112025244394	Bonafide Certificate	Mannem Pooja	krishna	451122733015	B.Tech	CSE	2022-2026
4	CERT/2025/46209	45112025561681	Scholarship Bonafide Certificate	Mannem Pooja	Krishnaiah	451122733015	B.Tech	CSE	2022-2026
5	CERT/2025/11934	45112025560837	Bonafide Certificate	Mannem Pooja	Krishna	451122733015	B.Tech	CSE	2022-2026
6	CERT/2025/63625	45112025344539	Study Conduct Certificate	Mannem Pooja	k	451122733015	B.Tech	CSE	2022-2026
7	CERT/2025/29020	45112025613600	Study Conduct Certificate	Mannem Pooja	krishna	451122733015	B.Tech	CSE	2022-2026
8	CERT/2025/51002	45112025310109	Study Conduct Certificate	Mannem Pooja	krishna	451122733015	B.Tech	CSE	2022-2026
9	CERT/2025/51184	45112025987652	Study Conduct Certificate	Mannem Pooja	krishna	451122733015	B.Tech	CSE	202-2026
10	CERT/2025/25816	45112025305315	Bonafide Certificate	Mannem Pooja	krishnaiah	451122733015	B.Tech	CSE	2022-2026
11	CERT/2025/70460	45112025352005	Bonafide Certificate	Mannem Pooja	svjbsb	451122733015	B.Tech	CSE	2022-2026
12	CERT/2025/36611	45112025283843	Scholarship Bonafide Certificate	Mannem Pooja	adhvbi	451122733015	B.Tech	CSE	2022-2026
13	CERT/2025/81638	45112025142900	Bonafide Certificate	Devarakonda Ashresh Kumar	Narendar	451122733008	B.Tech	CSE	2022-2026
14	CERT/2025/86372	45112025232492	Bonafide Certificate	Devarakonda Ashresh Kumar	Narendar	451122733008	B.Tech	CSE	2022-2026
15	CERT/2025/58143	45112025185604	Study Conduct Certificate	Devarakonda Ashresh Kumar	Narendar	451122733008	B.Tech	CSE	2022-2026
16	CERT/2025/41489	45112025838989	Bonafide Certificate	Mannem Pooja	Krishnaiah	451122733015	B.Tech	CSE	2022-2026
17	CERT/2025/64636	45112025752388	Bonafide Certificate	Mannem Pooja	Krishnaiah	451122733015	B.Tech	CSE	2022-2026
18	CERT/2025/27559	45112025352778	Bonafide Certificate	Mannem Pooja	krishna	451122733015	B.Tech	CSE	2022-2026
19	CERT/2025/15804	45112025697760	Bonafide Certificate	Mannem Pooja	Krishna	451122733015	B.Tech	CSE	2022-2026
20	CERT/2025/72073	45112025590376	Bonafide Certificate	Mannem Pooja	Krishna	451122733015	B.Tech	CSE	2022-2026
21	CERT/2025/36416	45112025863861	Study Conduct Certificate	Mannem Pooja	Krishna	451122733015	B.Tech	CSE	2022-2026
22	CERT/2025/89261	45112025796281	Bonafide Certificate	Mannem Pooja	Krishna	451122733015	B.Tech	CSE	2022-2026
23	CERT/2025/24910	45112025973609	Bonafide Certificate	Mannem Pooja	Krishna	451122733015	B.Tech	CSE	2022-2026
24	CERT/2025/61960	45112025344222	Bonafide Certificate	Mannem Pooja	krishna	451122733015	B.Tech	CSE	2022-2026
25	CERT/2025/65801	45112025954347	Bonafide Certificate	Mannem Pooja	Krishna	451122733015	B.Tech	CSE	2022-2026
26	CERT/2025/60648	45112025927896	Bonafide Certificate	Mannem Pooja	krishna	451122733015	B.Tech	CSE	2022-2026
27	CERT/2025/97795	45112025613831	Bonafide Certificate	Mannem Pooja	krishna	451122733015	B.Tech	CSE	2022-2026
28	CERT/2025/33207	45112025656662	Bonafide Certificate	Devarakonda Ashresh Kumar	Narendar	451122733008	B.Tech	CSE	2022-2026
29	CERT/2025/36368	45112025322466	Bonafide Certificate	Maddikuntla Likhitha	Saidulu	451122733022	B.Tech	CSE	2022-2026
30	CERT/2025/35331	45112025971424	Bonafide Certificate	Maddikuntla Likhitha	Maddikuntla Saidulu	451122733022	B.Tech	CSE	2022-2026
31	CERT/2025/50606	45112025780408	Study Conduct Certificate	Neelima Dogiparthi	Dogiparthi Ravikanth	451122733004	B.Tech	CSE	2022-2026
32	CERT/2025/76898	45112025622665	Scholarship Bonafide Certificate	Maddikuntla Likhitha	aaaaa	451122733022	B.Tech	CSE	2022-2026

id	qr_code	pdf_path	requested_date	issued_date	status	remark	created_at	updated_at	parent_prefix	present_year
1	[illegible]	45112025925863.pdf	2025-12-09 22:35:13	2025-12-09 17:09:11	verified		2025-12-09 22:35:13	2025-12-09 22:39:24	Mr.	IV
2	[illegible]	45112025387478.pdf	2025-12-10 12:47:41	2025-12-10 07:18:19	verified		2025-12-10 12:47:41	2025-12-10 12:48:29	Mr.	IV
3	[illegible]	45112025244394.pdf	2025-12-11 14:58:04	2025-12-11 09:29:04	verified		2025-12-11 14:58:04	2025-12-11 14:59:13	Mr.	IV
4	[illegible]	45112025561681.pdf	2025-12-11 15:46:38	2025-12-11 10:17:10	verified		2025-12-11 15:46:38	2025-12-11 15:47:16	Mr.	IV
5	[illegible]	45112025560837.pdf	2025-12-11 15:48:12	2025-12-11 10:18:58	verified		2025-12-11 15:48:12	2025-12-11 15:49:04	Mr.	IV
6	[illegible]	45112025344539.pdf	2025-12-11 15:48:25	2025-12-11 10:22:41	verified		2025-12-11 15:48:25	2025-12-11 15:52:46	Mr.	IV
7	[illegible]	45112025613600.pdf	2025-12-11 15:56:33	2025-12-11 10:30:24	verified	1	2025-12-11 15:56:33	2025-12-11 16:00:37	Mr.	IV
8	[illegible]	45112025310109.pdf	2025-12-11 15:56:48	2025-12-11 10:41:51	verified		2025-12-11 15:56:48	2025-12-11 16:12:09	Mr.	IV
9	[illegible]	45112025987652.pdf	2025-12-11 16:21:09	2025-12-11 10:51:29	verified		2025-12-11 16:21:09	2025-12-11 16:21:48	Mr.	IV
10	[illegible]	45112025305315.pdf	2025-12-11 16:26:12	2025-12-11 10:56:57	verified		2025-12-11 16:26:12	2025-12-11 16:27:17	Mr.	IV
11	[illegible]	45112025352005.pdf	2025-12-11 21:14:01	2025-12-11 15:49:17	verified		2025-12-11 21:14:01	2025-12-11 21:19:39	Mr.	IV
12	[illegible]	45112025283843.pdf	2025-12-11 21:14:19	2025-12-11 16:18:20	verified		2025-12-11 21:14:19	2025-12-11 21:48:40	Mr.	IV
13	[illegible]	45112025142900.pdf	2025-12-11 21:46:41	2025-12-11 16:17:18	verified		2025-12-11 21:46:41	2025-12-11 21:47:42	Mr.	IV
14	[illegible]	NULL	2025-12-15 12:10:58	NULL	pending	Duplicate request	2025-12-11 22:08:08	2025-12-15 17:40:58	Mr.	IV
15	[illegible]	45112025185604.pdf	2025-12-12 02:26:39	2025-12-11 20:57:11	verified		2025-12-12 02:26:39	2025-12-12 02:27:34	Mr.	IV
16	[illegible]	45112025838989.pdf	2025-12-12 02:47:42	2025-12-11 21:20:19	verified		2025-12-12 02:47:42	2025-12-12 02:50:39	Mr.	IV
17	[illegible]	45112025752388.pdf	2025-12-12 02:55:35	2025-12-11 21:26:02	verified		2025-12-12 02:55:35	2025-12-12 02:56:21	Mr.	IV
18	[illegible]	45112025352778.pdf	2025-12-14 11:57:13	2025-12-14 06:27:49	verified		2025-12-14 11:57:13	2025-12-14 11:58:00	Mr.	IV
19	[illegible]	45112025697760.pdf	2025-12-14 12:21:45	2025-12-14 06:52:10	verified		2025-12-14 12:21:45	2025-12-14 12:22:21	Mr.	IV
20	[illegible]	45112025590376.pdf	2025-12-14 17:33:45	2025-12-14 12:04:26	verified		2025-12-14 17:33:45	2025-12-14 17:34:48	Mr.	IV
21	[illegible]	45112025863861.pdf	2025-12-14 17:57:00	2025-12-14 12:27:51	verified		2025-12-14 17:57:00	2025-12-14 17:58:11	Mr.	IV
22	[illegible]	45112025796281.pdf	2025-12-14 17:57:27	2025-12-14 12:54:44	verified		2025-12-14 17:57:27	2025-12-14 18:25:03	Mr.	IV
23	[illegible]	NULL	2025-12-15 15:52:02	NULL	rejected	incorrect details	2025-12-15 15:52:02	2025-12-16 14:13:36	Mr.	IV
24	[illegible]	45112025344222.pdf	2025-12-15 15:57:09	2025-12-15 10:29:42	verified		2025-12-15 15:57:09	2025-12-15 16:00:04	Mr.	IV
25	[illegible]	45112025954347.pdf	2025-12-15 16:00:49	2025-12-15 10:34:24	verified		2025-12-15 16:00:49	2025-12-15 16:04:44	Mr.	IV
26	[illegible]	NULL	2025-12-15 16:12:58	NULL	rejected	Invalid Details	2025-12-15 16:12:58	2025-12-16 12:33:02	Mr.	IV
27	[illegible]	45112025613831.pdf	2025-12-15 17:26:24	2025-12-16 07:02:37	verified		2025-12-15 17:26:24	2025-12-16 12:32:46	Mr.	IV
28	[illegible]	NULL	2025-12-15 12:41:44	NULL	pending	Invalid Details	2025-12-15 17:42:52	2025-12-15 18:11:44	Mr.	IV
29	[illegible]	45112025322466.pdf	2025-12-16 12:12:50	2025-12-16 06:43:38	verified		2025-12-16 12:12:50	2025-12-16 12:13:51	Mr.	IV
30	[illegible]	NULL	2025-12-16 07:25:45	NULL	pending	Duplicate request	2025-12-16 12:42:10	2025-12-16 12:55:45	Mr.	IV
31	[illegible]	45112025780408.pdf	2025-12-16 14:01:30	2025-12-16 08:39:15	verified		2025-12-16 14:01:30	2025-12-16 14:09:34	Mr.	IV
32	[illegible]	45112025622665.pdf	2025-12-16 15:31:31	2025-12-16 10:02:04	verified		2025-12-16 15:31:31	2025-12-16 15:32:22	Mr.	IV

32 rows in set (0.01 sec)

Figure 6.3.4: certificates Data

- **otp_records (id, email, otp_code, purpose, expires_at, is_used, created_at)**

mysql> desc otps;

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
email	varchar(255)	YES		NULL	
otp	varchar(255)	NO		NULL	
created_at	datetime	NO		CURRENT_TIMESTAMP	DEFAULT_GENERATED
expires_at	datetime	NO		NULL	

5 rows select * from otps;

Figure 6.3.5: Describe otps

`mysql> select * from otps;`

id	email	otp	created_at	expires_at
87	ashreshdevarakonda@gmail.com	4956cbd20efbdf41066afef9a15aafe0:51ef4e5350a1581e23849aa028e11fb7	2025-12-16 17:48:57	2025-12-16 12:28:57

`1 row in set (0.00 sec)`

Figure 6.3.6: otps Data

- **qr_verification_logs (id, certificate_id, reference_num, verification_time, ip_address, user_agent, is_valid, error_message, created_at)**

`mysql> desc qr_verification_logs;`

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
reference_num	varchar(20)	YES		NULL	
user_agent	text	YES		NULL	
ip_address	varchar(45)	YES		NULL	
verified_at	timestamp	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED

`5 rows in set (0.03 sec)`

Figure 6.3.7: Describe qr_verification_logs

`mysql> select * from qr_verification_logs;`

id	reference_num	user_agent	ip_address	verified_at
1	45112025352778	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-14 12:16:51
2	45112025352778	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-14 12:16:51
3	45112025352778	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-14 12:19:54
4	45112025352778	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-14 12:19:54
5	45112025352778	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-14 12:20:02
6	45112025352778	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-14 12:20:02
7	45112025352778	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-14 12:20:02
8	45112025352778	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-14 12:20:02
9	45112025352778	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-14 12:20:25
10	45112025352778	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-14 12:20:25
11	45112025352778	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-14 12:23:00
12	45112025352778	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-14 12:23:00
13	45112025352778	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-14 12:23:00
14	45112025352778	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-14 12:23:00
15	45112025697760	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-14 12:23:16
16	45112025697760	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-14 12:23:16
17	45112025352778	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-14 12:26:18
18	45112025352778	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-14 12:26:18
19	45112025352778	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-14 12:34:47
20	45112025352778	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-14 12:34:48
21	45112025352778	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-14 12:37:46
22	45112025352778	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-14 12:38:02
23	45112025352778	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-14 12:38:07
24	45112025352778	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-14 12:38:10
25	45112025352778	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-14 12:38:16
26	45112025352778	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-14 12:38:21
27	45112025352778	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-14 12:38:24
28	45112025352778	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-14 12:39:19
29	45112025352778	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-14 12:39:19
30	45112025352778	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-14 12:40:16
31	45112025352778	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-14 12:40:17
32	45112025352778	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-14 12:41:17
33	45112025352778	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-14 12:45:14
34	45112025352778	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-14 12:45:25
35	45112025352778	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-14 12:45:30
36	45112025352778	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-14 12:47:28
37	45112025352778	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-14 12:47:28
38	45112025352778	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-14 12:55:20
39	45112025352778	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-14 12:55:20
40	45112025352778	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-14 17:24:08
41	45112025352778	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-14 17:24:08
42	45112025352778	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-14 17:24:08
43	45112025352778	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-14 17:24:31
44	45112025352778	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-14 17:24:31
45	45112025863861	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-14 18:00:23
46	45112025796281	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-14 18:28:18
47	45112025796281	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-15 15:04:09
48	45112025796281	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::1	2025-12-15 15:04:09
49	45112025954347	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::ffff:172.16.36.47	2025-12-15 16:45:51
50	45112025954347	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::ffff:172.16.36.47	2025-12-15 16:45:51
51	45112025322466	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::ffff:172.16.36.47	2025-12-16 12:15:35
52	45112025322466	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::ffff:172.16.36.47	2025-12-16 12:25:39
53	45112025322466	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::ffff:172.16.36.47	2025-12-16 12:27:07
54	45112025322466	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::ffff:172.16.36.47	2025-12-16 13:10:41
55	45112025780408	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::ffff:172.16.36.47	2025-12-16 14:10:27
56	45112025622665	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::ffff:172.16.36.47	2025-12-16 15:32:56
57	45112025142900	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::ffff:172.16.36.47	2025-12-16 22:58:02
58	45112025142900	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::ffff:172.16.36.47	2025-12-16 22:58:02
59	45112025142900	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::ffff:172.16.36.47	2025-12-16 23:00:16
60	45112025142900	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36 Edg/143.0.0.0	::ffff:172.16.36.47	2025-12-16 23:00:16

`60 rows in set (0.03 sec)`

Figure 6.3.8: qr_verification_logs Data

6.4 Component Interaction Flow

Component interaction can be summarized as follows:

- **Registration Flow:**
  Student submits registration details → backend validates and sends OTP → student verifies OTP → user record created in database.
- **Certificate Request Flow:**
  Student submits request → backend inserts record with status "pending" → admin dashboard shows the request.
- **Approval and Generation Flow:**
  Admin approves → backend generates reference number, QR code, and PDF → database updated → notification sent → student can download.
- **Verification Flow:**
  Public user scans QR code or enters reference number → backend fetches certificate → displays validity and logs verification attempt.

7. IMPLEMENTATION DETAILS

7.1 Backend Implementation (Node.js & Express)

The backend is structured into modules for configuration, routes, controllers, services, and middleware. Configuration files handle database connections and environment variables. Routes map API endpoints to corresponding controllers, which implement business logic using services.

Key middleware components handle JSON parsing, CORS, sessions, authentication checks, error handling, and input validation. Passwords are hashed with bcrypt before insert or update operations. All database interactions use parameterized queries to prevent SQL injection.

The backend module is built with Express.js and organized as follows:

```
backend/
├── src/
│   ├── server.js
│   ├── config/
│   │   ├── db.js
│   │   └── mailer.js
│   ├── controllers/
│   │   ├── authController.js
│   │   ├── certificateController.js
│   │   ├── templateController.js
│   │   └── userController.js
│   ├── middlewares/
│   │   ├── errorHandler.js
│   │   └── validateOtp.js
│   └── routes/
│       ├── auth.js
│       ├── certificate.js
│       ├── templates.js
│       └── user.js
├── public/
│       ├──assets
│       │    └── signatures/
│       ├── certificates/
│       └── qr-codes/
├── templates/
```

```
│       ├──bonafide_certifcate.html
│       ├──conduct_certifcate.html
│       └──scholarship-bonafide-certifcate.html
└── package.json
```

7.2 Core Backend Modules

7.2.1 Authentication Module:

Implements registration, login, logout, password reset, and OTP verification. It ensures that only verified users can access protected routes.

- **Registration**: Validates email, sends OTP, verifies OTP, creates user with hashed password.
- **Login**: Checks credentials, sets session.
- **Logout**: Destroys session.
- **Password Reset**: Sends OTP, verifies, updates password.
- **OTP Verification**: Encrypts/decrypts OTPs, checks expiry.
- **Protected Routes**: Uses session-based auth with middleware.

7.2.2 Certificate Module:

Handles creation of certificate request records, retrieval of student requests, listing pending requests for admins, and approval or rejection. On approval, it triggers QR and PDF generation and updates certificate status.

- **Apply**: Students submit requests with details.
- **Retrieve**: Lists for students/admins.
- **Approve/Reject**: Admins update status; approval generates PDF and sends email.
- **Status Tracking**: Pending, verified, rejected.

7.2.3 Template Module:

Provides APIs to create, update, activate, and retrieve certificate templates. Templates are stored with their HTML and CSS content in the database.

- **Storage**: HTML/CSS in DB or files
- **Template Type**: Automatically selects the template based on student application.
- **Type of Templates:**

1. Bonafide Certificate
2. Study Conduct Certificate
3. Scholarship Bonafide Certificate

College Logo

UNIVERSITY COLLEGE OF ENGINEERING & TECHNOLOGY
MAHATMA GANDHI UNIVERSITY
Nalgonda - 508254, T.S., India

Sl. No: {{SL_NO}}
Date: {{DATE}}
Roll No: {{ROLL_NO}}

QR Code

BONAFIDE CERTIFICATE

This is to certify that {{STUDENT_NAME}}, {{SO_DO}} {{PARENT_NAME}} is a bonafide student of this College. {{HE_SHE}} is presently studying {{BRANCH}} {{COURSE}}, {{PRESENT_YEAR}} year during the Academic year {{ACADEMIC_YEAR}} in University College of Engineering & Technology, Mahatma Gandhi University, Nalgonda. {{HE_SHE}} has maintained satisfactory attendance and academic progress.

This certificate is issued for official use and verification of student enrollment status.

HoD Signature
HoD Signature

Principal Signature
Principal Signature

Printed by University College of Engineering & Technology, Mahatma Gandhi University, Nalgonda. | www.mgunalgonda.com

Figure 7.2.3.1: Bonafide Certificate Template

College Logo

UNIVERSITY COLLEGE OF ENGINEERING & TECHNOLOGY
MAHATMA GANDHI UNIVERSITY
Nalgonda - 508254, T.S., India

Sl. No: {{SL_NO}}

Date: {{DATE}}

Roll No: {{ROLL_NO}}

QR Code

CONDUCT CERTIFICATE

This is to certify that {{STUDENT_NAME}}, {{SO_DO}} {{PARENT_NAME}} is a student of this College. {{HE_SHE}} is presently studying {{BRANCH}} {{COURSE}}, {{PRESENT_YEAR}} year during the Academic year {{ACADEMIC_YEAR}} in University College of Engineering & Technology, Mahatma Gandhi University, Nalgonda. {{HE_SHE}} has demonstrated exemplary conduct, discipline, and good moral character. Reference ID: {{REFERENCE_NUM}}

This certificate is issued to certify the commendable behavior and conduct of the student.

HoD Signature

HoD Signature

Principal Signature

Principal Signature

Printed by University College of Engineering & Technology, Mahatma Gandhi University, Nalgonda. | www.mgunalgonda.com

Figure 7.2.3.2: Conduct Certificate Template

UNIVERSITY COLLEGE OF ENGINEERING & TECHNOLOGY
MAHATMA GANDHI UNIVERSITY
Nalgonda - 508254, T.S., India

Sl. No: {{SL_NO}}

Date: {{DATE}}

Roll No: {{ROLL_NO}}

BONAFIDE CERTIFICATE

This is to certify that {{STUDENT_NAME}}, {{SO_DO}} {{PARENT_NAME}} is a bonafide student of this College. {{HE_SHE}} is presently studying {{BRANCH}} {{COURSE}}, {{PRESENT_YEAR}} year during the Academic year {{ACADEMIC_YEAR}} in University College of Engineering & Technology, Mahatma Gandhi University, Nalgonda. Reference ID: {{REFERENCE_NUM}}

This certificate is issued for the purpose of submitting to scholarship.

Candidate Signature **HoD Signature** **Principal Signature**

Printed by University College of Engineering & Technology, Mahatma Gandhi University, Nalgonda. | www.mgunalgonda.com

Figure 7.2.3.3: Scholarship Bonafide Certificate Template

7.2.4 OTP & Email Service Module:

Generates random OTPs, encrypts them, stores them in the database, and sends them to the user via email using an SMTP service such as Gmail.

- **OTP Generation**: Random 6-digit, encrypted storage
- **Email Sending**: Via Nodemailer with Gmail SMTP
- **Expiry**: 10 minutes
- **Encryption**: Uses crypto for OTPs

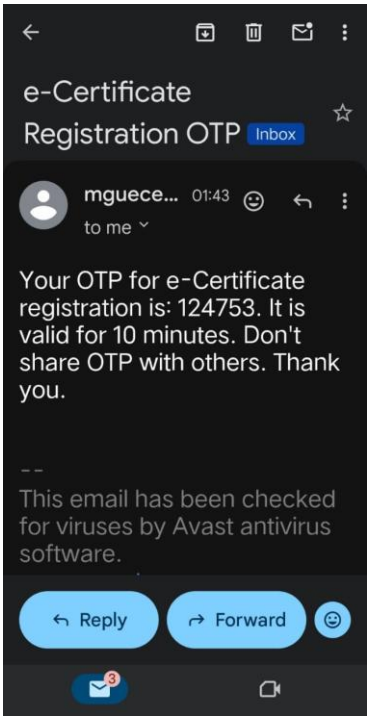

Figure 7.2.4.1: Email Otp Sending

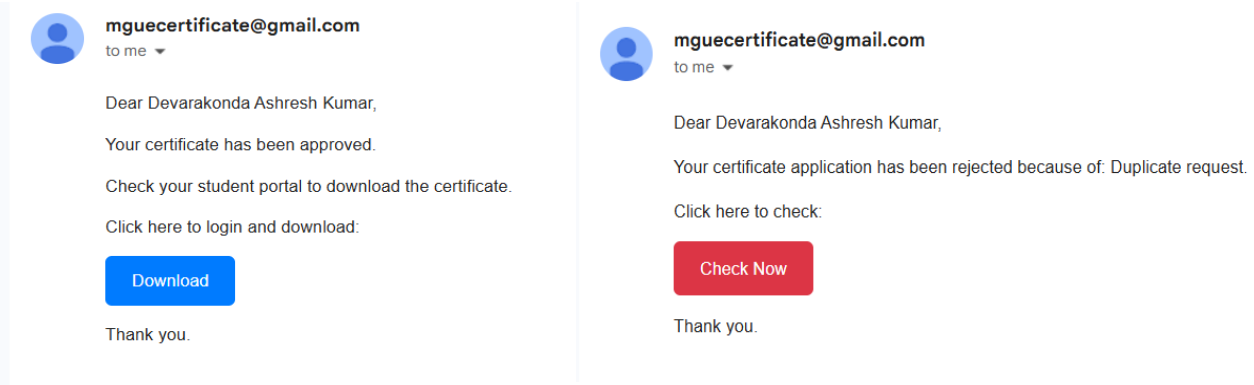

Figure 7.2.4.2: Email Notification Service

7.2.5 QR & PDF Generation Services:

Use external libraries to generate QR code images and render certificate content into PDF files, saving them on the server and storing paths in the database.

- **QR Generation**: Uses qrcode library, saves as PNG.
- **PDF Generation**: Uses pdfkit, renders HTML templates into PDFs.
- **Storage**: Files in qr-codes and certificates, paths in DB.

7.3 Frontend Implementation (React)

The React frontend is organized into pages and reusable components, with React Router handling navigation between views. Authentication state is stored using context or local state so that protected routes only render when the user is logged in. Axios or fetch is used to call backend APIs.

Student and admin dashboards are implemented as layout components containing navigation sidebars and main content areas. Individual sections such as “Apply Certificate”, “My Requests”, “Pending Requests”, or “Issued Certificates” are separate components that fetch and display relevant data.

The frontend module is built with react.js with .css and organized as follows:

```
frontend/
├── .gitignore
├── build/
│   ├── asset-manifest.json
│   ├── index.html
│   ├── manifest.json
│   └── static/
│       ├── css/
│       ├── js/
│       └── media/
├── node_modules/
├── package-lock.json
├── package.json
├── public/
│   ├── index.html
│   └── manifest.json
├── README.md
└── src/
├── App.css
```

```
├── App.jsx
├── index.css
├── index.jsx
├── admindashboard/
│   ├── AdminDash.css
│   ├── AdminDash.jsx
│   ├── CertificateRequests.jsx
│   ├── ChangePassword.jsx
│   ├── IssuedCertificates.jsx
│   ├── QRVerificationLogs.jsx
│   ├── RequestDetails.jsx
│   ├── StudentDetails.jsx
│   └── StudentManagement.jsx
├── components/
│   ├── ProtectedRoute.jsx
│   ├── ProtectedRouteAdmin.jsx
│   └── ProtectedRouteStudent.jsx
├── forgotpassword/
│   ├── ForgotPass.css
│   ├── ForgotPass.jsx
│   ├── newpassword/
│   │   ├── NewPass.css
│   │   └── NewPass.jsx
│   └── passwordresetsuccess/
│   └── PasswordResetSuccess.jsx
├── hooks/
│   └── useSessionTimeout.js
├── login/
│   └── Login.jsx
├── pages/
│   ├── VerifyCertificate.jsx
│   └── VerifyQuickly.css
├── register/
│   ├── createpassword/
│   │   ├── CreatePass.css
│   │   └── CreatePass.jsx
│   ├── Register.css
│   ├── Register.jsx
│   └── registeredsuccess/
│   └── RegisteredSuccess.jsx
├── studentdashboard/
```

```
│   ├── ApplyCertificate.jsx
│   ├── CertificateStatus.jsx
│   ├── CertificateView.jsx
│   ├── ChangePassword.jsx
│   ├── DownloadCertificates.jsx
│   ├── StudDash.css
│   ├── StudDash.jsx
│   └── StudentProfile.jsx
└── styles/
    └── shared.css
```

7.3.1 Register Page (src/register/Register.jsx)

Page Description

The Register page manages new user onboarding with an OTP-based verification flow, where users submit email and role, receive an OTP, verify it, and then proceed to password creation.

Logic

- **State Management:** Maintains email, role, otp, and step to control the multi-step registration process.
- **API Interactions:** Invokes POST /api/register/send-otp to dispatch the OTP and POST /api/register/verify-otp to validate the code and create the account.
- **Validation:** Ensures correct email format and enforces a 6-digit OTP constraint.

Key Components and Features

- Step-based UI containing email input, OTP entry, and verification views.
- Direct navigation links to the Login page for existing users.
- Styled via Register.css and exposed as a public route that leads into CreatePass.jsx.
- Depends on Axios for API calls and useNavigate for routing, integrating with the email service for OTP delivery.

Key Functions:

```
const sendOtp = async () => {

 const res = await axios.post('/api/register/send-otp', { email, role });
```

```
  if (res.status === 200) setStep('verify');
};
const verifyOtp = async () => {
  const res = await axios.post('/api/register/verify-otp', { email, otp, password });
  if (res.status === 200) navigate('/register-success');
};
```

7.3.2 Login Page (src/login/Login.jsx)

Page Description:

The Login page is the entry point for both students and administrators, providing authentication through email, password, and role selection. On successful login, users are redirected to their respective dashboards, with clear error handling for invalid credentials.

Logic

- **State Management:** Utilizes useState hooks to manage email, password, role, loading, and error states.
- **API Interactions:** Sends a POST /api/login request using Axios; on success, stores the JWT token in localStorage and navigates via useNavigate.
- **Validation:** Performs client-side checks for mandatory fields and relies on server responses for invalid-credential errors.

Key Components and Features

- Form elements including <input> fields for email and password, a <select> for role, and a submit <button>.
- Conditional rendering of a loading spinner and error messages.
- Global responsive styling through App.css, with routing protection to redirect if the user is already logged in.

- Uses Axios for HTTP calls and React Router for navigation, and provides a “Forgot Password?” link to initiate the reset flow.

Key Functions:

```
const handleSubmit = async (e) => {
 e.preventDefault();
 setLoading(true);
 try {
  const res = await axios.post('/api/login', { email, password });
  localStorage.setItem('token', res.data.token);
  navigate(role === 'admin' ? '/admin' : '/student');
 } catch (err) {
  setError('Invalid credentials');
 } finally {
  setLoading(false);
 }
};
```

7.3.3 Create Password Page (src/register/createpassword/CreatePass.jsx)

Page Description

The Create Password page allows users who have successfully verified their OTP to define a strong account password and confirm it before completion of registration.

Logic

- **State Management:** Tracks password, confirmPassword, and validation errors.
- **API Interactions:** Submits the chosen password to the backend, which persists the hash in the database.

- **Validation:** Uses regular expressions to enforce minimum length, numeric and symbol requirements, and checks for password match.

Key Components and Features

- Dual password input fields with optional visibility toggle and inline error messages.
- Styled using CreatePass.css and integrated into the registration routing sequence.
- Relies on React hooks for internal state, with additional server-side validation at save time.

Key Functions:

```
const handleSubmit = (e) => {
  e.preventDefault();
  if (password !== confirmPassword) setErrors('Passwords do not match');
  else if (!/^(?=.*\d)(?=.*[@#$%^&*!]).{8,}$/.test(password)) setErrors('Weak password');
  else navigate('/register-success'); };
```

7.3.4 Register Success Page (src/register/registeredsuccess/RegisteredSuccess.jsx)

Page Description

This confirmation page is displayed after a successful registration, informing the user that the account has been created and providing a direct link to the login screen.

Logic

- Maintains only minimal state for static content and uses useNavigate to redirect to /login on button click.

Key Components and Features

- A success icon, explanatory message, and a “Go to Login” button.
- Simple CSS styling and routing as the terminal step of the registration flow, with no API interactions.

7.3.5 Forgot Password Page (src/forgotpassword/ForgotPass.jsx)

Page Description

The Forgot Password page enables users to initiate a password reset by submitting their registered email, receiving an OTP, and validating it to continue.

Logic

- **State Management:** Controls email, otp, and step for a guided reset sequence.
- **API Interactions:** Uses POST /api/forgot-password/send-otp to send the OTP and POST /api/auth/reset-password to verify and progress.
- **Validation:** Ensures non-empty and properly formatted email input along with a valid 6-digit OTP.

Key Components and Features

- Email entry form, OTP input, and reset-flow navigation toward NewPass.jsx.
- Styled via ForgotPass.css, exposed as a public route, and implemented with Axios for backend communication.
- Reuses logic patterns from the registration OTP flow for consistency.

7.3.6 New Password Page (src/forgotpassword/newpassword/NewPass.jsx)

Page Description

The New Password page allows users who have verified their reset OTP to define and confirm a new account password.

Logic

- Reuses the validation and state-handling principles from CreatePass.jsx, including strength checks and match verification, before submitting to the backend.

Key Components and Features

- Password and confirm-password fields with a submit action.
- Styled with NewPass.css and placed within the password-reset routing pipeline, navigating to a success page after completion.

7.3.7 Password Reset Success Page (src/forgotpassword/passwordresetsuccess/PasswordResetSuccess.jsx)

Page Description

This page confirms that the password reset operation has completed successfully and guides the user back to the login screen.

Logic

- Contains minimal logic, primarily using useNavigate to route back to /login when requested.

Key Components and Features

- Static success message, confirmation icon, and login link.
- Basic styling and routing as the terminal step of the reset flow, without any backend calls.

7.3.8 Admin Dashboard (src/admindashboard/AdminDash.jsx)

Page Description

The Admin Dashboard serves as the primary interface for administrators, providing tabbed access to certificate requests, issued certificates, verification logs, and user management.

Logic

- **State Management:** Manages activeTab and authenticated admin data.
- **API Interactions:** Fetches necessary summary data on mount and on tab changes.
- **Validation:** Utilizes ProtectedRouteAdmin to ensure only admin-role users can access the dashboard.

Key Components and Features

- Navigation bar and tab system that swaps between CertificateRequests, IssuedCertificates, and related components.
- Styled with AdminDash.css, routed under /admin, and dependent on React Router for nested navigation and logout handling.

7.3.9 Certificate Requests (src/admindashboard/CertificateRequests.jsx)

Page Description

The Certificate Requests view lists all pending certificate requests awaiting administrative approval or rejection.

Logic

- **State Management:** Maintains a requests array and loading/error flags.
- **API Interactions:** Uses GET /api/certificates/pending to fetch requests and additional endpoints to approve or reject entries.

Key Components and Features

- Tabular layout showing student details, certificate type, date, and action buttons.
- Inherits styling from the admin layout and is rendered as a sub-component within the Admin Dashboard, updating row status based on API responses.

7.3.10 Issued Certificates (src/admindashboard/IssuedCertificates.jsx)

Page Description

This page provides administrators with a list of all approved certificates, supporting viewing and downloading of generated PDFs.

Logic

- **State Management:** Holds a certificates collection and associated UI state.
- **API Interactions:** Calls GET /api/certificates/issued to load data and uses navigation handlers (such as handleView(id)) to open detailed certificate views.

Key Components and Features

- Table representation including reference number, student name, issue date, and a View action.
- Custom table styling and routing integration to /certificate/:id, ensuring admin authorization before access.

7.3.11 Other Admin Sub-Components (e.g., QRVerificationLogs.jsx, StudentManagement.jsx)

Page Description

Additional admin views include QR Verification Logs for monitoring scans and Student Management for handling user accounts.

Logic

- Manage local state for log or user lists and perform CRUD-style API operations to fetch, update, and delete records.

Key Components and Features

- Data tables with filters, edit forms, and action buttons for administrative tasks.
- Share a consistent layout with the dashboard and rely on secure API endpoints for updates.

7.3.12 Student Dashboard (src/studentdashboard/StudDash.jsx)

Page Description

The Student Dashboard functions as the central workspace for students, offering access to certificate application, status tracking, downloading, and profile management.

Logic

- Mirrors the admin dashboard's tabbed structure but enforces student role checks and fetches data specific to the logged-in user.

Key Components and Features

- Tabs such as ApplyCertificate, CertificateStatus, and related utilities.
- Styled using StudDash.css, routed via /student, and integrated with protected routes for authenticated students only.

7.3.13 Apply Certificate (src/studentdashboard/ApplyCertificate.jsx)

Page Description

The Apply Certificate page presents a form where students can request new certificates and optionally upload required supporting documents.

Logic

- **State Management:** Tracks form fields, file inputs, and validation messages.
- **API Interactions:** Submits a POST /api/certificates/submit request using FormData to handle text and file payloads.
- **Validation:** Verifies mandatory fields, file size, and allowed MIME types before submission.

Key Components and Features

- Structured form with text inputs, dropdowns, file upload control, and submit button.
- Dedicated form styling and close coupling with backend Multer middleware for file handling.

7.3.14 Certificate Status (src/studentdashboard/CertificateStatus.jsx)

Page Description

The Certificate Status page displays all certificate requests submitted by the current student along with their current processing states.

Logic

- Consumes the "my requests" API to fetch the student's requests and maps them into a status view, refreshing as needed.

Key Components and Features

- List or table with columns for certificate type, submission date, and status (pending, approved, rejected).
- Visual design similar to the Issued Certificates view for consistency.

7.3.15 Other Student Sub-Components (e.g. DownloadCertificates.jsx, StudentProfile.jsx)

Page Description

Additional student pages enable downloading of approved certificate PDFs and editing of personal profile data.

Logic

- Use API calls to retrieve downloadable certificate metadata and to update profile details securely.

Key Components and Features

- Buttons for direct PDF download and forms for updating profile fields.
- Incorporate client-side validation and integrate with backend endpoints for persistence.

7.3.16 Certificate View (src/studentdashboard/CertificateView.jsx)

Page Description

The Certificate View component provides an embedded viewer for individual certificate PDFs within the application interface.

Logic

- **State Management:** Holds the resolved PDF URL corresponding to the selected certificate.
- **API Interactions:** Requests the PDF resource from the backend and validates that the user is authorized to access it.

Key Components and Features

- An <iframe> element that renders the PDF file and a print button for browser-based printing.
- Dedicated viewer styling and route binding under /certificate/:id, leveraging backend static file serving.

7.3.17 Verify Certificate (src/pages/VerifyCertificate.jsx)

Page Description

The Verify Certificate page is publicly accessible and supports verification of certificates using a reference number, typically reached via QR code scan.

Logic

- **State Management:** Manages reference_num, certificate details, loading, and error flags.
- **API Interactions:** Sends GET /api/verify/:ref requests to validate the certificate and returns corresponding metadata.
- **Validation:** Ensures that the supplied reference number follows expected format rules before calling the API.

Key Components and Features

- Input control for manual reference entry, result display card showing certificate details, and action buttons for retry or printing.
- Styled using VerifyQuickly.css, routed under /verify/:ref, and designed to operate without authentication for external verifiers.

7.4 API Design

The backend exposes RESTful APIs with clear resource-based endpoints. For example:

- POST /api/register, POST /api/login, POST /api/logout
- POST /api/certificates/submit, GET /api/certificates/my-requests
- GET /api/certificates/pending, POST /api/certificates/:id/approve, POST /api/certificates/:id/reject
- GET /api/certificates/:id/download
- GET /api/verify/:reference_num

Responses follow a consistent JSON structure with status, message, and data fields, making it easier for frontend components to handle success and error states.

8. TESTING & QUALITY ASSURANCE

8.1 Testing Methodologies

The system is tested using a combination of white-box and black-box techniques. White-box tests focus on internal logic such as password hashing, OTP encryption/decryption, and certificate generation functions. Black-box tests cover end-to-end user flows like registration, login, certificate request, approval, download, and verification without reference to internal code.

8.2 Testing Types and Details

8.2.1 Functional Testing

Functional testing verified that all features work as specified, including user registration, certificate application, approval/rejection, and downloads. This ensures the system meets its core requirements and user workflows are complete.

Key Points:

- User Registration: Tested OTP-based registration with email validation and database insertion.
- Certificate Application: Validated form submission, business rules (e.g., year restrictions), and duplicate prevention.
- Admin Approval/Rejection: Confirmed status updates, PDF/QR generation, and email notifications.
- Downloads and Verification: Ensured PDF downloads and QR code scanning work correctly.
- Edge Cases: Tested invalid inputs, missing data, and error scenarios.

8.2.2 Integration Testing

Integration testing examined interactions between frontend, backend, database, and external services (email, PDF generation). This ensures seamless data flow and service dependencies.

Key Points:

- Frontend-Backend Communication: Verified API calls for authentication, certificate CRUD, and file uploads.

- Database Integration: Tested queries, joins, and data consistency across tables (users, certificates, otps).
- Email Service: Confirmed Nodemailer integration with Gmail SMTP for OTP and notification emails.
- PDF/QR Generation: Validated Puppeteer/html-pdf libraries with QRCode for certificate creation.
- Error Handling: Ensured failures in one component don't break the entire flow.

8.2.3 Security Testing

Security testing assessed authentication, authorization, input validation, and protection against common vulnerabilities like SQL injection and XSS. This protects user data and prevents unauthorized access.

Key Points:

- Authentication Security: Tested bcrypt password hashing, session management, and OTP encryption.
- Authorization: Verified role-based access (student vs. admin) and protected routes.
- Input Validation: Checked parameterized queries to prevent SQL injection and frontend sanitization for XSS.
- Session Security: Ensured HttpOnly cookies and secure flags in production.
- Data Protection: Confirmed sensitive data (passwords, OTPs) are not logged or exposed.

8.2.4 Performance Testing

Performance testing measured response times for API calls, PDF generation, and email sending under normal load. This identifies bottlenecks and ensures acceptable user experience.

Key Points:

- API Response Times: Measured endpoints like login, certificate application, and stats retrieval (<2 seconds).
- PDF Generation: Timed certificate creation using Puppeteer (30-40 seconds for A4 PDFs).
- Email Sending: Evaluated OTP delivery via Gmail SMTP (5-10 seconds average).
- Concurrent Users: Tested with 5 simultaneous users to check stability.

- Resource Usage: Monitored CPU/memory during PDF generation and database queries.

8.2.5 Usability Testing

Usability testing evaluated user interface intuitiveness, error messages, and mobile responsiveness. This ensures the system is user-friendly and accessible.

Key Points:

- Interface Design: Assessed form layouts, navigation, and dashboard clarity.
- Error Messages: Verified clear, helpful feedback for validation failures and system errors.
- Mobile Responsiveness: Tested on various screen sizes using Bootstrap CSS.
- Workflow Simplicity: Ensured registration to download flow is intuitive with minimal steps.
- Accessibility: Checked basic WCAG compliance (alt texts, keyboard navigation).

8.2.6 Compatibility Testing

Compatibility testing ensured functionality across Chrome and Firefox browsers. This guarantees consistent behavior across supported platforms.

Key Points:

- Browser Support: Tested core features (forms, downloads, QR scanning) in Chrome 120.x and Firefox 121.x.
- Rendering Consistency: Verified PDF display, email links, and UI elements across browsers.
- JavaScript Compatibility: Ensured React components and Axios requests work without issues.
- Device Testing: Included desktop and mobile views for responsiveness.
- Version Compatibility: Confirmed with Node.js 18.x, MySQL 8.0, and specified npm packages.

8.2.7 Unit Testing

Unit tests are written for core backend functions, including:

- User registration and login functions (valid and invalid input).
- OTP generation, encryption, storage, and validation.

- Certificate reference number generation and QR/PDF generation functions.

Frontend components can also be tested to ensure they render correctly and handle state transitions when API calls succeed or fail.

8.3 Quality Factors

Quality assurance evaluates aspects such as performance, reliability, usability, and security.

- **Performance:** Acceptable response times and quick certificate generation.
- **Reliability:** Stable operation without crashes or data loss; proper error handling.
- **Usability:** Clear, user-friendly interfaces and meaningful error messages.
- **Security:** Correct implementation of authentication, authorization, and input validation; no critical vulnerabilities found in basic security review.

9. INSTALLATION & EXECUTION

9.1 Prerequisites

Before installation, the following must be available:

- Node.js and npm installed on the host machine.
- MySQL server running with administrative credentials.
- A Gmail or institutional SMTP account for sending OTP and notification emails.
- Git for cloning the source code repository.

9.2 Recommended Development Tools

Recommended VS Code Extensions:

- ESLint – JavaScript/React linting and code quality enforcement.
- Prettier – Automatic formatting for JS, JSON, and Markdown.
- React Extension Pack – React snippets, IntelliSense, and debugging tools.
- MySQL Extension – Database connection and query execution inside VS Code.
- GitLens – Advanced Git history, blame, and branch visualization.
- Thunder Client – Lightweight REST API client for testing backend endpoints.

9.3 Backend Setup and Core Libraries

Backend Setup

- Open the project folder ecerti.
- Navigate to the backend folder and run npm install to install dependencies.
- Create a .env file with database, session, and email configuration values.
- Use MySQL to create the project database and execute migration scripts.
- Create directories for storing certificates and QR code images (for example, public/certificates and public/qr-codes).

Backend Core Libraries

- Express.js (v4.19.2) – REST API framework and routing.
- MySQL (v2.18.1) – MySQL driver with connection pooling.
- Bcrypt (v6.0.0) – Secure password hashing.
- Nodemailer (v6.9.13) – Email sending for OTP and notifications.

- Puppeteer (v21.3.8) – Headless browser for HTML-to-PDF rendering.
- QRCode (v1.5.3) – QR code generation for verification links.
- Multer (v1.4.5-lts.1) – File upload handling for requests.
- CORS (v2.8.5) – Cross-origin resource sharing between frontend and backend.
- Dotenv (v16.4.5) – Environment variable loading from .env.
- Express-Session (v1.18.2) – Session-based authentication storage.
- PDFKit (v0.17.2) – Programmatic PDF generation.
- HTML-PDF (v3.0.1) – HTML to PDF conversion used with Puppeteer.

```
* Executing task: node src/server.js

Server is running on port 5000
Connected to MySQL database
[POST] /api/login => { email: 'ashresh_devarakonda', password: 'DAshresh@1974' }
User logged in. Session user: {
  id: 2,
  username: 'ashresh_devarakonda',
  email: 'dashreshkumar@gmail.com',
  roll_number: '451122733008',
  role: 0
}
isAuthenticated middleware triggered for path: /user/details
req.sessionID: pje9JLvRSgj6oG0_z6IqjwISPaW_IjmU
req.session.user: {
  id: 2,
  username: 'ashresh_devarakonda',
  email: 'dashreshkumar@gmail.com',
  roll_number: '451122733008',
  role: 0
}
isAuthenticated middleware triggered for path: /user/details
req.sessionID: pje9JLvRSgj6oG0_z6IqjwISPaW_IjmU
req.session.user: {
  id: 2,
  username: 'ashresh_devarakonda',
  email: 'dashreshkumar@gmail.com',
  roll_number: '451122733008',
  role: 0
}
```

Figure 9.3: Backend Execution

9.4 Frontend Setup and Core Libraries

Frontend Setup

- Navigate to the frontend folder and run npm install to install React and other dependencies.
- Configure the API base URL (for example, http://localhost:5000/api or http://172.16.36.47:5000/api) in a config file or .env.
- Run npm start to launch the React development server on port 3000.

Frontend Core Libraries

- React (v18.2.0) and React-DOM (v18.3.1) – Component-based UI and rendering.
- React Router DOM (v7.9.4) – Client-side routing between views.
- Axios (v1.13.2) – HTTP client for backend API communication.
- React Icons (v5.5.0) – Icon set for UI components.
- FontAwesome (v7.1.0, v3.1.0) – Additional icon packs.
- HTML2Canvas (v1.4.1) and jsPDF (v2.5.1) – Generate PDFs from HTML views.
- Web Vitals (v2.1.4) – Track frontend performance metrics.
- Testing libraries (@testing-library/react, etc., v16.3.0) – Unit and integration testing of React components.

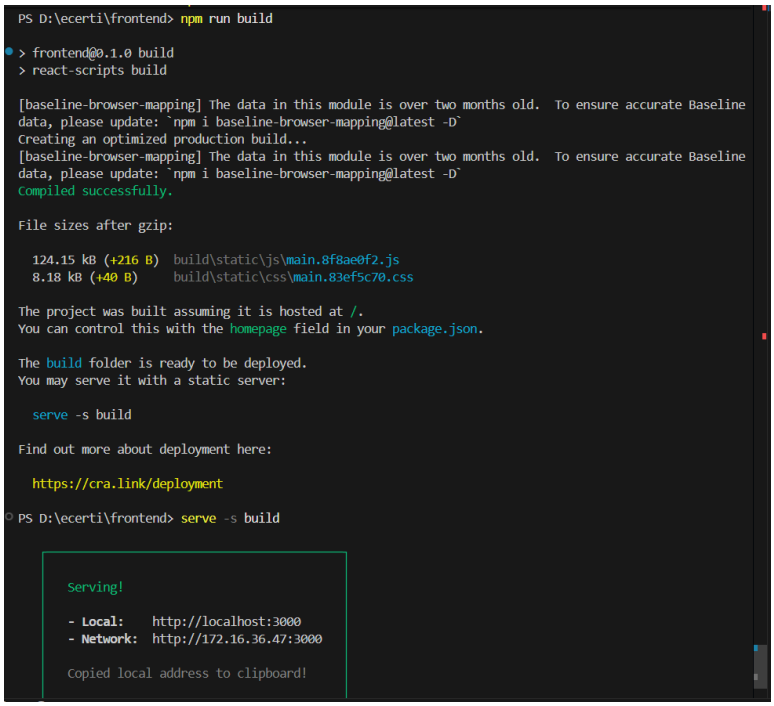

Figure 9.4: Frontend Execution

9.5 Database Setup

Technology: MySQL (relational database)

Connection: Managed via db.js, which uses the mysql npm package and loads credentials from .env.

Environment (.env file)

- DB_HOST=localhost
- DB_USER=username
- DB_PASSWORD=password
- DB_DATABASE=ecertificate
- EMAIL_USER=mgucertificate@gmail.com
- EMAIL_PASS=app-password
- PORT=5000
- DB_DATABASE=http://10.55.47.47:3000

Connection Code

```
require('dotenv').config({ path: require('path').resolve(__dirname, '../../.env') });
const mysql = require('mysql');
const util = require('util');
const db = mysql.createConnection({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_DATABASE,
});
db.connect((err) => {
  if (err) {
    console.error('Error connecting to MySQL:', err);
    return;
```

```
  }
  console.log('Connected to MySQL database');
});
db.query = util.promisify(db.query);
module.exports = db;
```

```
Default
Enter password: *************
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 17
Server version: 8.0.43 MySQL Community Server - GPL

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| ecertificate       |
| information_schema |
| mysql              |
| performance_schema |
| sakila             |
| sys                |
| world              |
+--------------------+
7 rows in set (0.00 sec)

mysql> use ecertificate;
Database changed
mysql> show tables;
+-----------------------+
| Tables_in_ecertificate |
+-----------------------+
| certificates          |
| otps                  |
| qr_verification_logs  |
| users                 |
+-----------------------+
4 rows in set (0.02 sec)
```

Figure 9.5: Database and tables

9.6 Running the Application

- Ensure the MySQL server is running and the database is accessible.
- Start the backend server with node src/server.js (or nodemon).
- Start the frontend server with npm start in the frontend directory.
- Open a browser and navigate to http://localhost:3000 or http://172.16.36.47:3000 to access the application.
- Optionally, use Postman or Thunder Client to test API endpoints directly.

10. Deployment

10.1 Overview

This section provides instructions for deploying the e-Certificate Management System locally on your Windows PC using VS Code. The deployment runs all components (backend, frontend, database) on your machine for development or testing purposes. No external servers, GitHub, or cloud platforms are required. All steps are executed in the VS Code integrated terminal.

Local Deployment Architecture:

- **Backend**: Node.js server running on http://localhost:5000.
- **Frontend**: React app running on http://localhost:3000.
- **Database**: MySQL running locally on port 3306.
- **Services**: Local file storage for PDFs/QR codes, Gmail SMTP for emails.

Assumptions:

- VS Code is installed with integrated terminal.
- Project is cloned or available in a local folder (e.g., ecerti).
- Windows 10/11 with administrative access.

10.2 Prerequisites

1. **Install Required Software (if not already installed):**
   - Node.js: Download from nodejs.org and install. Verify: node --version (should be 18.x).
   - MySQL: Download MySQL Installer from mysql.com and install. Set root password during setup.
   - Git: Install from git-scm.com for version control.
2. **VS Code Extensions:**
   - Install "Remote SSH" (for future remote access, optional).
   - Ensure terminal is set to PowerShell or Command Prompt.
3. **Project Setup:**
   - Open the project folder in VS Code: File > Open Folder > D:\ecerti.
   - Open integrated terminal: Ctrl + (backtick).

10.3 Database Setup

1. **Start MySQL Service:**
   - Open VS Code terminal and run: net start mysql (or use Windows Services to start MySQL80).
   - Log in to MySQL: mysql -u root -p (enter your root password).
2. **Create Database:**
   - CREATE DATABASE ecertificate;
   - EXIT;
3. **Import Schema:**
   - In VS Code terminal, navigate to project: cd D:\ecerti\docs.
   - Run SQL scripts manually or create a file schema.sql
   - Import: mysql -u root -p ecertificate < schema.sql.
4. **Update Configuration:**
   - Edit .env in VS Code:

     DB_HOST=localhost

     DB_USER=root

     DB_PASSWORD=your_mysql_root_password

     DB_DATABASE=ecertificate

     PORT=5000

     FRONTEND_URL=http://localhost:3000

     EMAIL_USER=your-gmail@gmail.com

     EMAIL_PASS=your-gmail-app-password

     SESSION_SECRET=your-secure-secret

10.4 Backend Deployment

1. Install Dependencies:

- In VS Code terminal: cd backend && npm install.

2. Start the Server:

- Run: npm start.
- Expected output: "Server is running on port 5000 and bound to 0.0.0.0. Connected to MySQL database."

- Access API at http://localhost:5000/api/health (if implemented).

3. Run in Background (Optional):

- Use a tool like pm2 (install globally: npm install -g pm2).
- Start: pm2 start src/server.js --name ecerti-backend.
- Check status: pm2 list.

10.5 Frontend Deployment

1. Install Dependencies:

- In VS Code terminal: cd frontend && npm install.

2. Start Development Server:

- Run: npm start.
- Expected output: "Compiled successfully! You can now view frontend in the browser."
- Access app at http://localhost:3000.

3. Build for Production (Optional):

- Run: npm run build.
- Serve locally: Install serve globally (npm install -g serve), then serve -s build -l 3001.
- Access at http://localhost:3001.

10.6 File Storage Setup

1. Directories: Ensure these exist (VS Code will create them automatically):

- backend\public\certificates\
- backend\public\qr-codes\
- backend\public\assets\ (add logo/signature images here).

2. Permissions: No special setup needed on Windows for local access.

10.7 Testing Deployment

1. Full System Test:

- Open browser to http://localhost:3000.

- Register a user, verify OTP, apply for certificate.
- Switch to admin role (manually update DB), approve certificate.
- Check email, download PDF, scan QR.

2. Monitor Logs:

- **Backend**: Check VS Code terminal output.
- **Frontend**: Open DevTools (F12) for console errors.
- **Database**: Query logs in MySQL.

3. Health Checks:

- API: curl http://localhost:5000/api/certificate/stats (should return JSON).
- Frontend: Ensure no 404 errors.

10.8 Maintenance and Troubleshooting

1. Stop Services:

- Backend: Ctrl + C in terminal or pm2 stop ecerti-backend.
- Frontend: Ctrl + C.
- Database: net stop mysql.

2. Updates:

- Pull changes: git pull (if using Git).
- Reinstall: npm install in backend/frontend.
- Restart services.

3. Common Issues:

- Port Conflicts: Check if 3000/5000 are free: netstat -ano | findstr :3000.
- MySQL Connection: Ensure service is running; check password in .env.
- Email Errors: Verify Gmail settings; use app password.
- Build Errors: Clear cache: npm cache clean --force.
- CORS Issues: Ensure FRONTEND_URL is http://localhost:3000.

4. Backup:

- Database: mysqldump -u root -p ecertificate > backup.sql.
- Files: Copy backend\public\ folders.

5. Performance:

- Monitor CPU/RAM via Task Manager.
- For heavy load, consider upgrading hardware.

This local deployment allows full testing and development on your PC. For production, migrate to a server using similar steps. If issues occur, check VS Code terminal output and ensure all prerequisites are met. Document any custom configurations for future reference.#### 10.4 Backend Deployment

1. Install Dependencies:

- In VS Code terminal: cd backend && npm install.

2. Start the Server:

- Run: npm start.
- Expected output: "Server is running on port 5000. Connected to MySQL database."
- Access API at http://localhost:5000/api/health (if implemented).

3. Run in Background (Optional):

- Use a tool like pm2 (install globally: npm install -g pm2).
- Start: pm2 start src/server.js --name ecerti-backend.
- Check status: pm2 list.

11. USER GUIDES

11.1 Student User Guide

- **Registration and Login:**
  Students open the application URL, choose “Register”, fill in their details, receive an OTP via email, verify it, and set a password. After registration, they log in using their email and password.

- **Profile Management:**
  Update personal details in the "Profile" section if needed.

- **Applying for Certificates:**
  From the student dashboard, they navigate to the “Apply Certificate” section, select the type of certificate (e.g., Bonafide, Conduct), specify course and academic year, and submit the request.

- **Tracking Status and Downloading:**
  The “My Requests” section shows all requests with statuses such as Pending, Approved, or Rejected. For approved requests, students can download the certificate as a PDF from the “Download Certificates” section.

- **Change Password:**
  Navigate to the "Change Password" section in the student dashboard. Enter your current password, new password (must be 8+ characters with numbers/symbols), and confirm. Submit to update securely. You'll receive a confirmation message upon success.

11.2 Administrator Guide

- **Logging In:**
  Administrators log in using credentials created by the system or stored in the database initially.

- **Managing Requests:**
  The “Certificate Requests” section lists all pending requests with student details. Admins can review each request, select a template, and either approve or reject it with comments.

- **Viewing Issued Certificates and Logs:**

The “Issued Certificates” section displays all generated certificates with reference numbers and issue dates. The “Verification Logs” section shows QR and reference-based verification attempts for auditing.

- **Student Management:**
  The “Student management” section displays all student details and available of view the data of each student using view button.

11.3 Public Verification Guide

- **QR-Based Verification:**
  Employers or institutions receiving the certificate scan the QR code using a smartphone camera. The browser opens a verification URL displaying certificate details and validity status.

- **Manual Verification:**
  In case the QR code cannot be scanned, verifiers can visit the public verification page, enter the reference number printed on the certificate, and view the verification result.

12. RESULTS & ANALYSIS

12.1 Output Screens and Snapshots

Representative screenshots illustrate key parts of the system:

- Login and registration pages.
- Student dashboard, certificate request form, and status tracking table.
- Admin dashboard showing pending requests, approval screens, and issued certificates.
- Sample generated certificate PDF with layout, student details, reference number, and QR code.
- Public verification page showing valid or invalid certificate messages.

12.2 Login and Registration pages

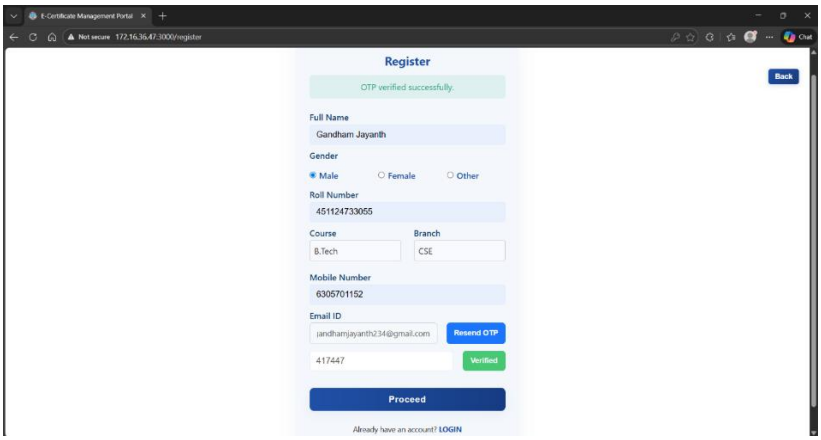

Figure 12.2.1: Registration page

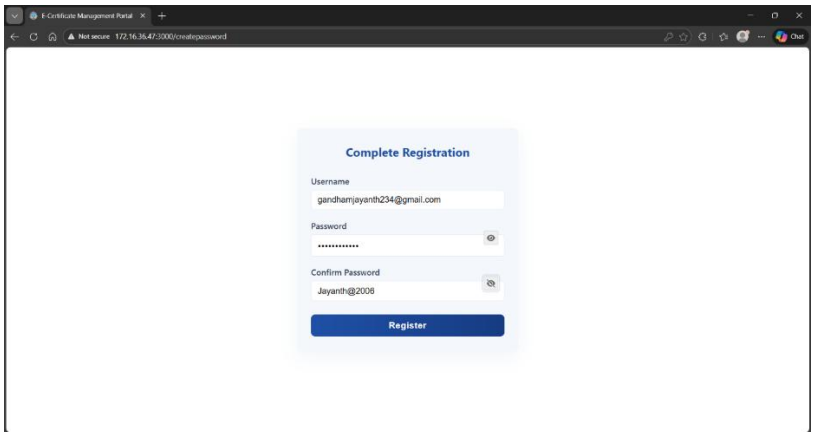

Figure 12.2.2: Create Password page

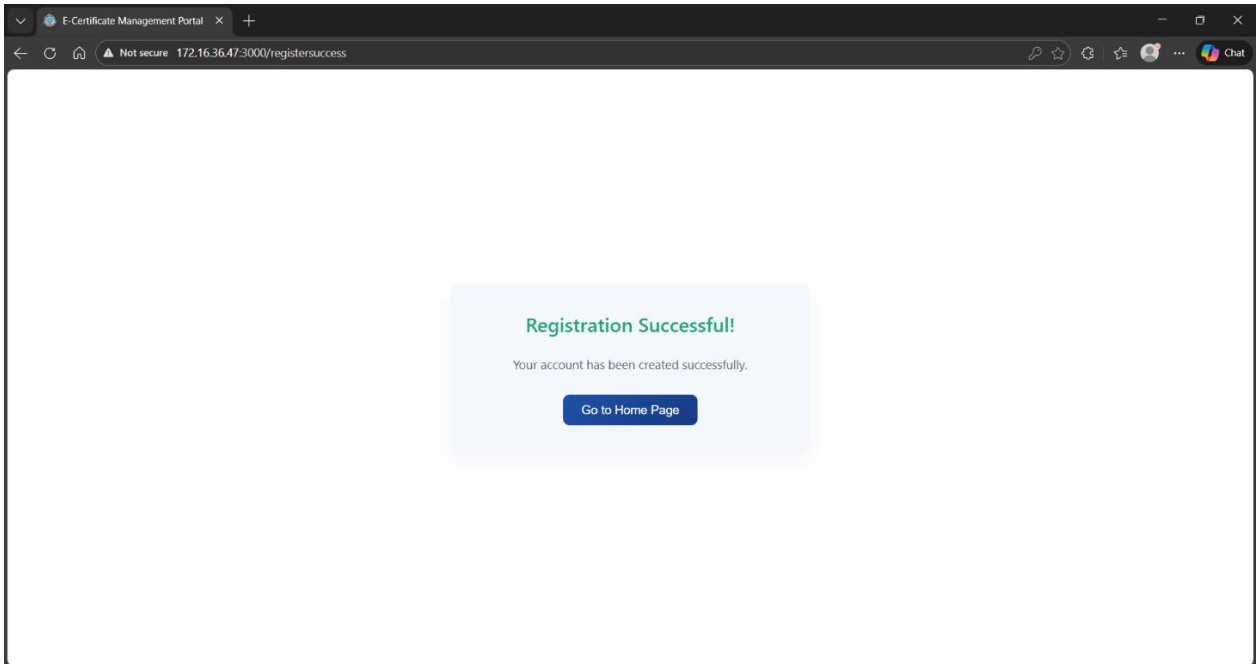

Figure 12.2.3: Registration Successful page

12.3 Forgot Password Pages

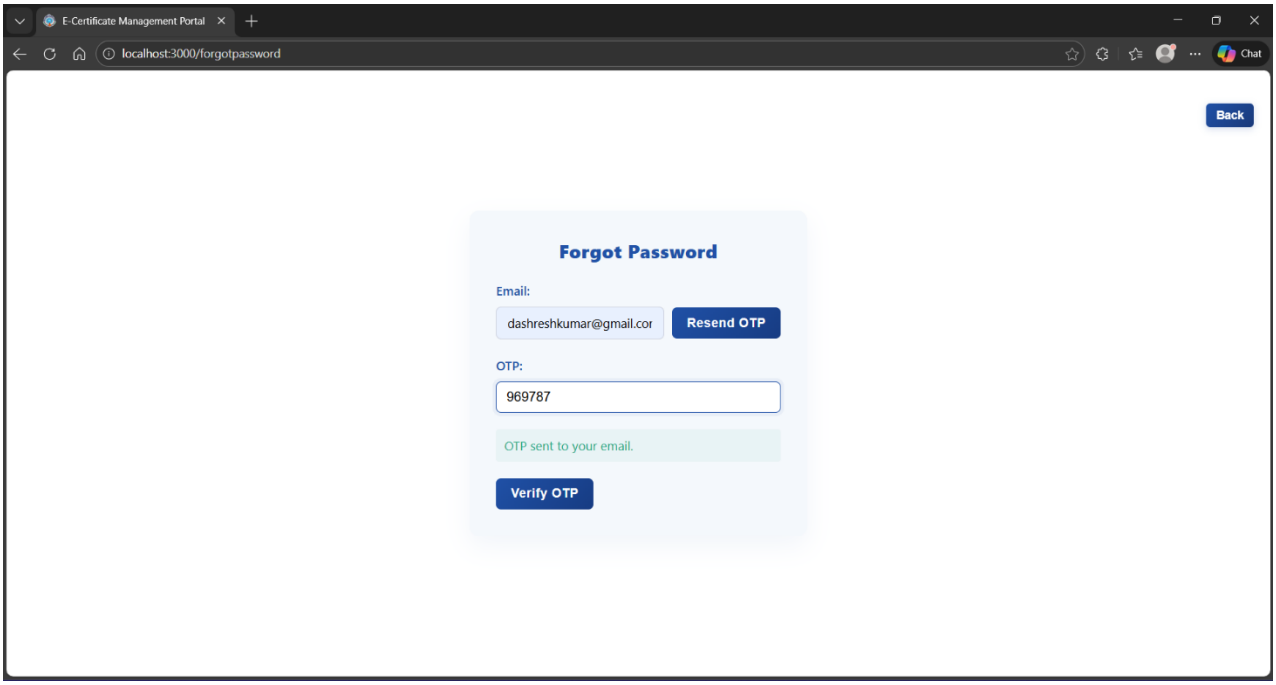

Figure 12.3.1: Forgot Password page

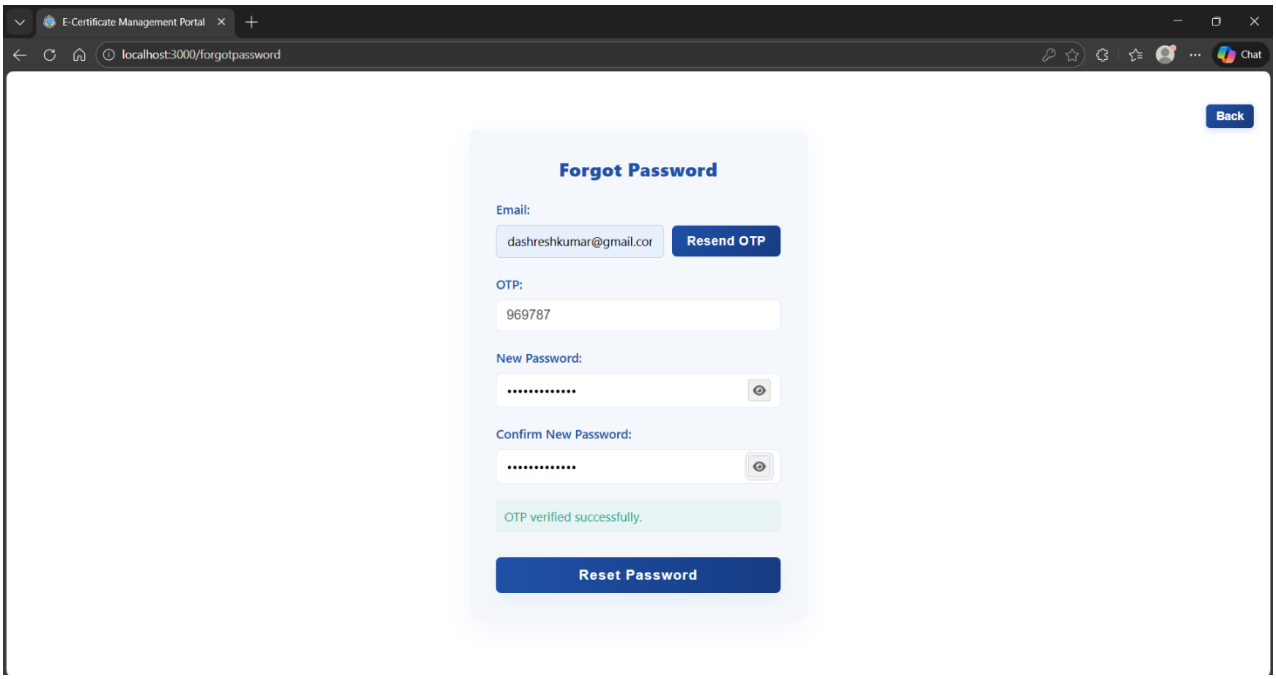

Figure 12.3.2: Create New Password page

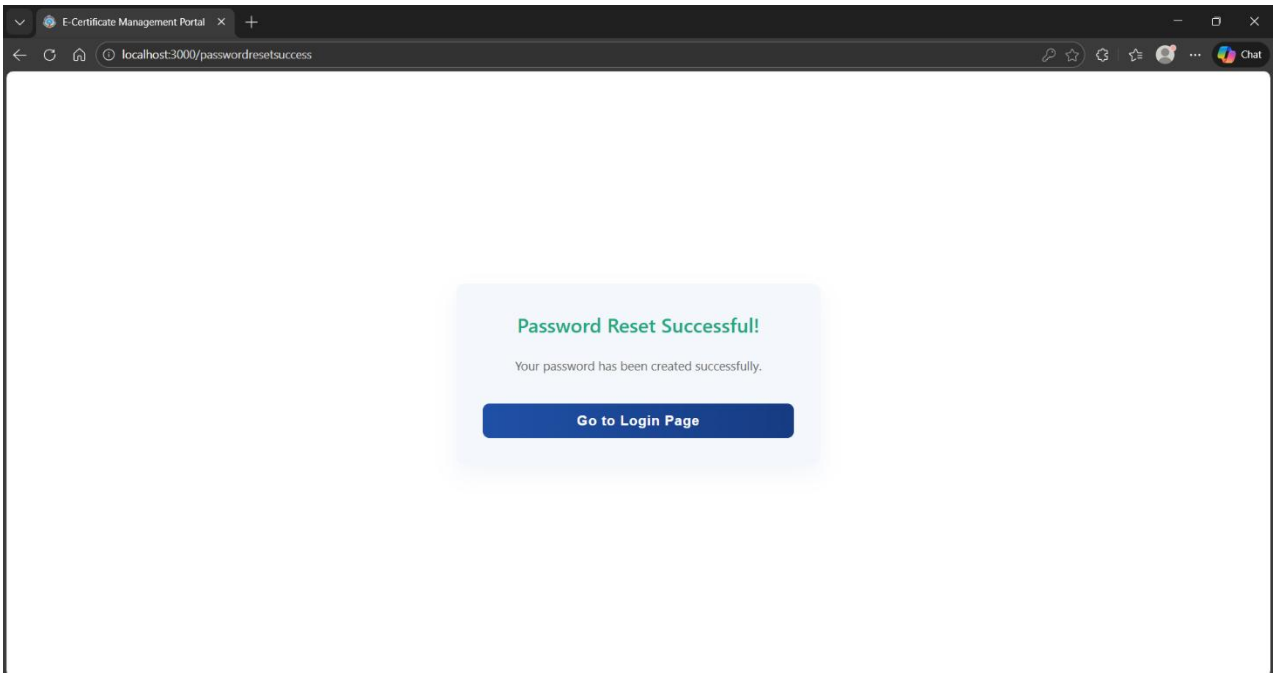

Figure 12.3.3: New Password Successful page

12.4 Student Dashboards

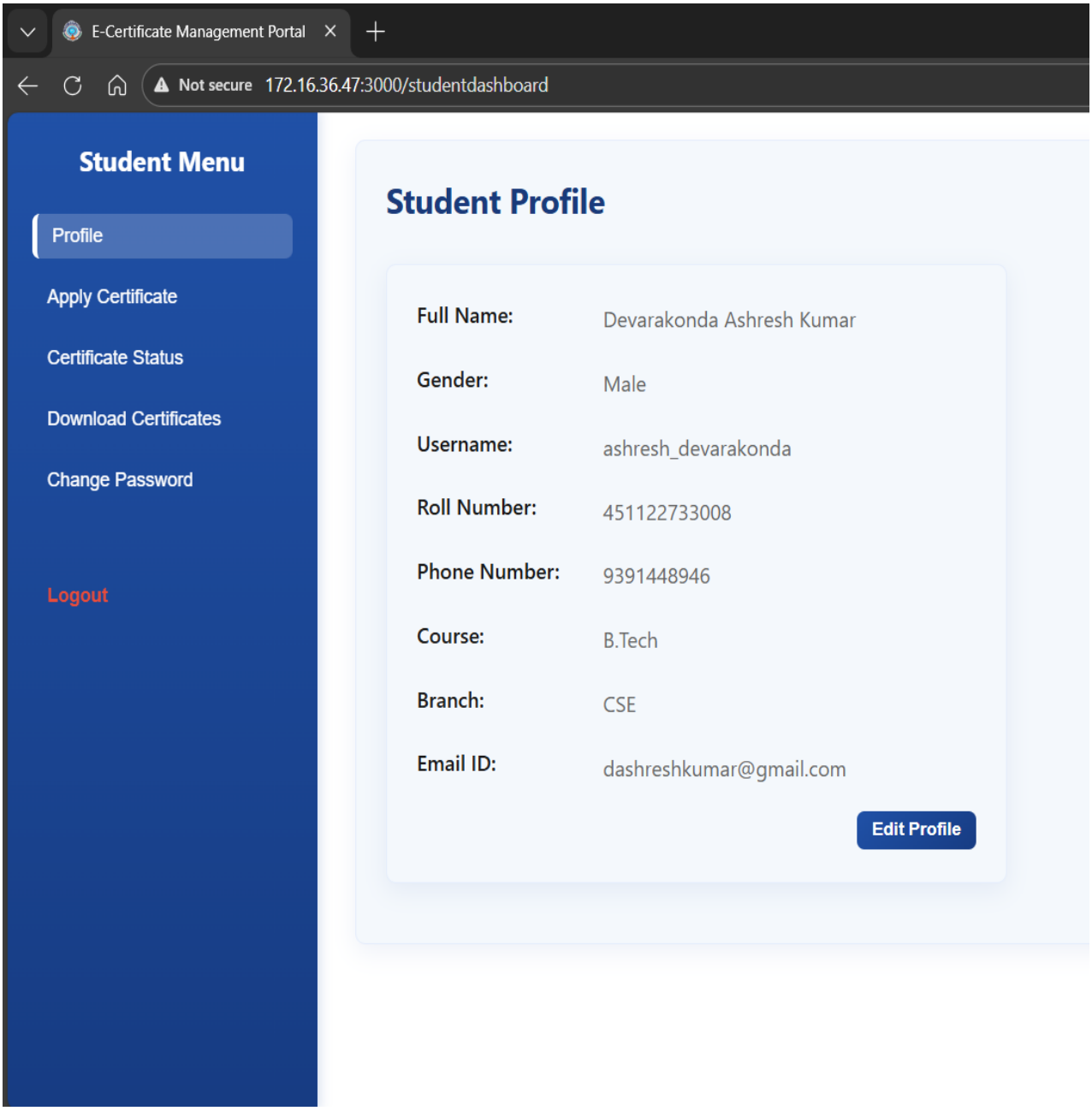

Figure 12.4.1: Student Profile page

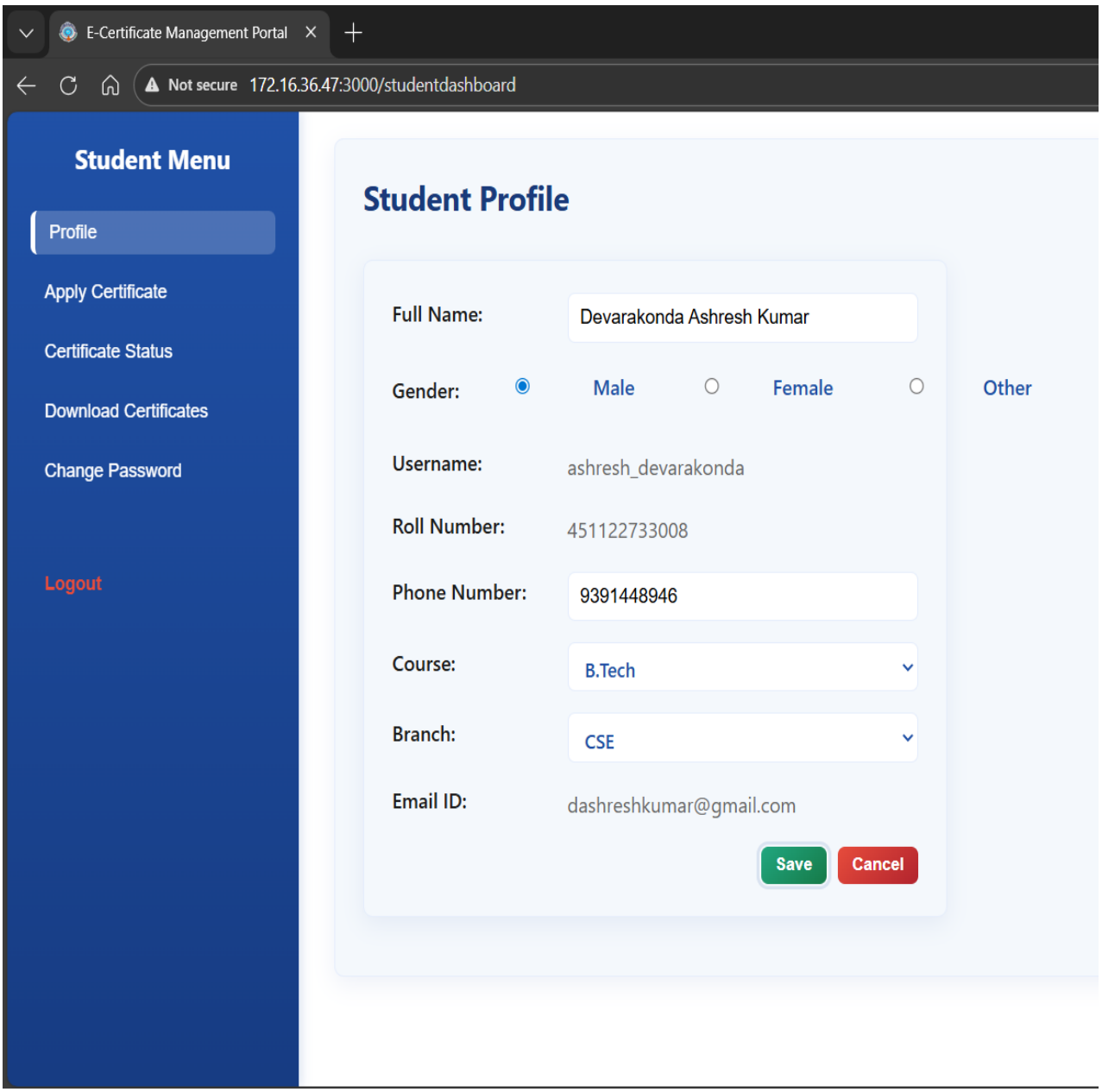

Figure 12.4.2: Student Profile editable page

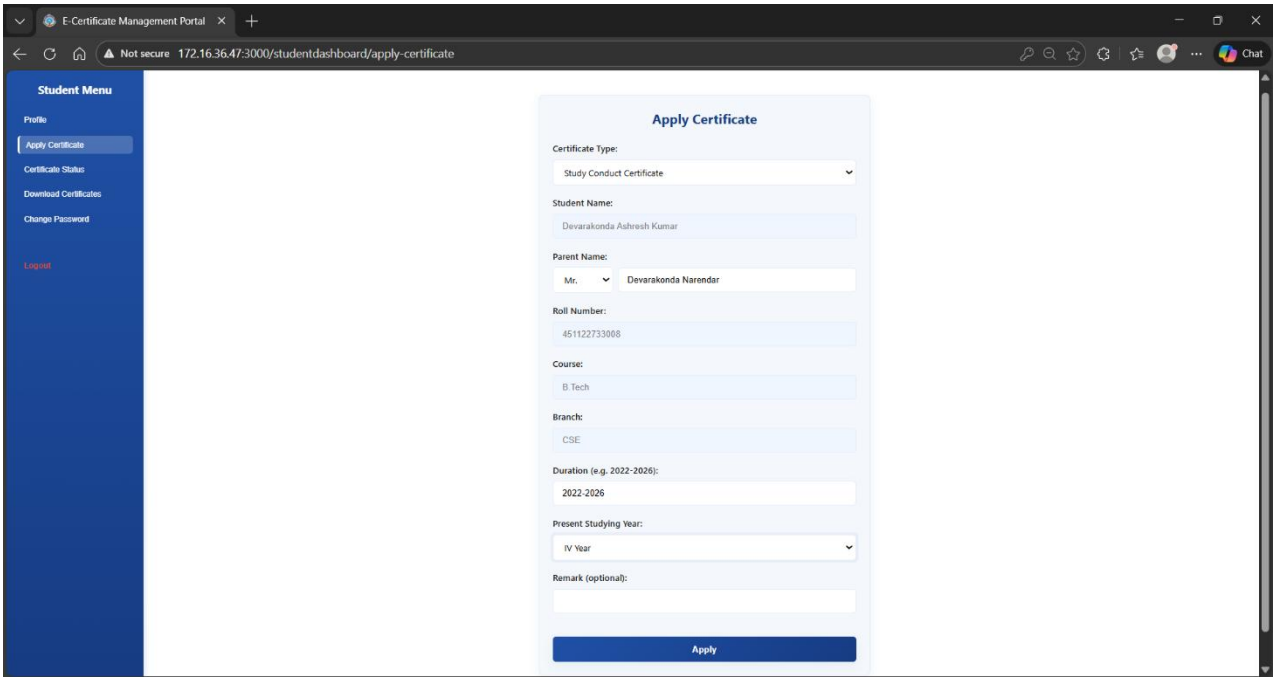

Figure 12.4.3: Apply Certificate page

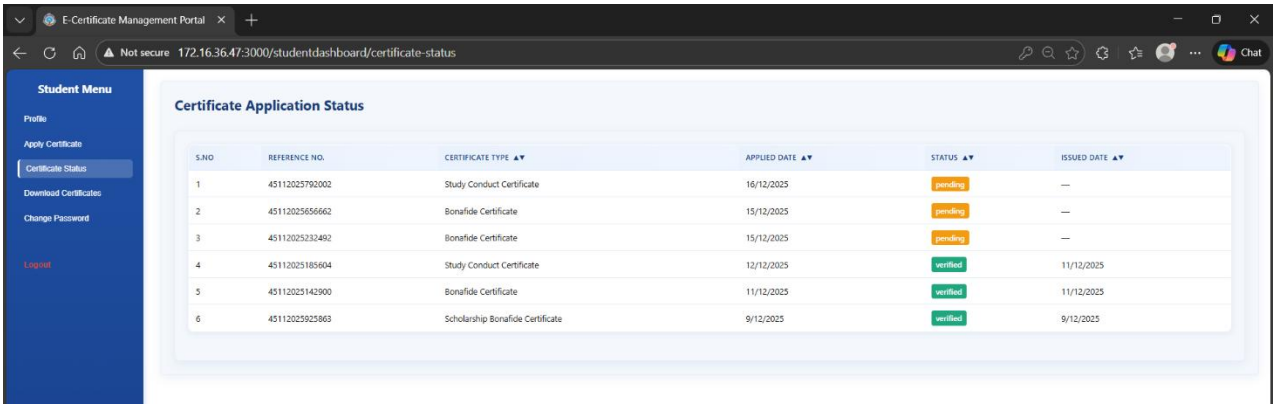

Figure 12.4.4: Certificate Status page

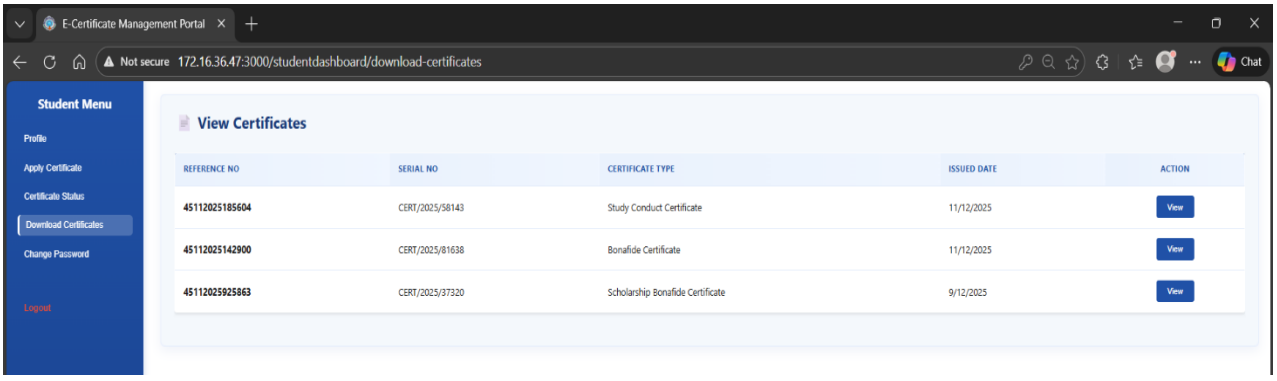

Figure 12.4.5: Download Certificate page

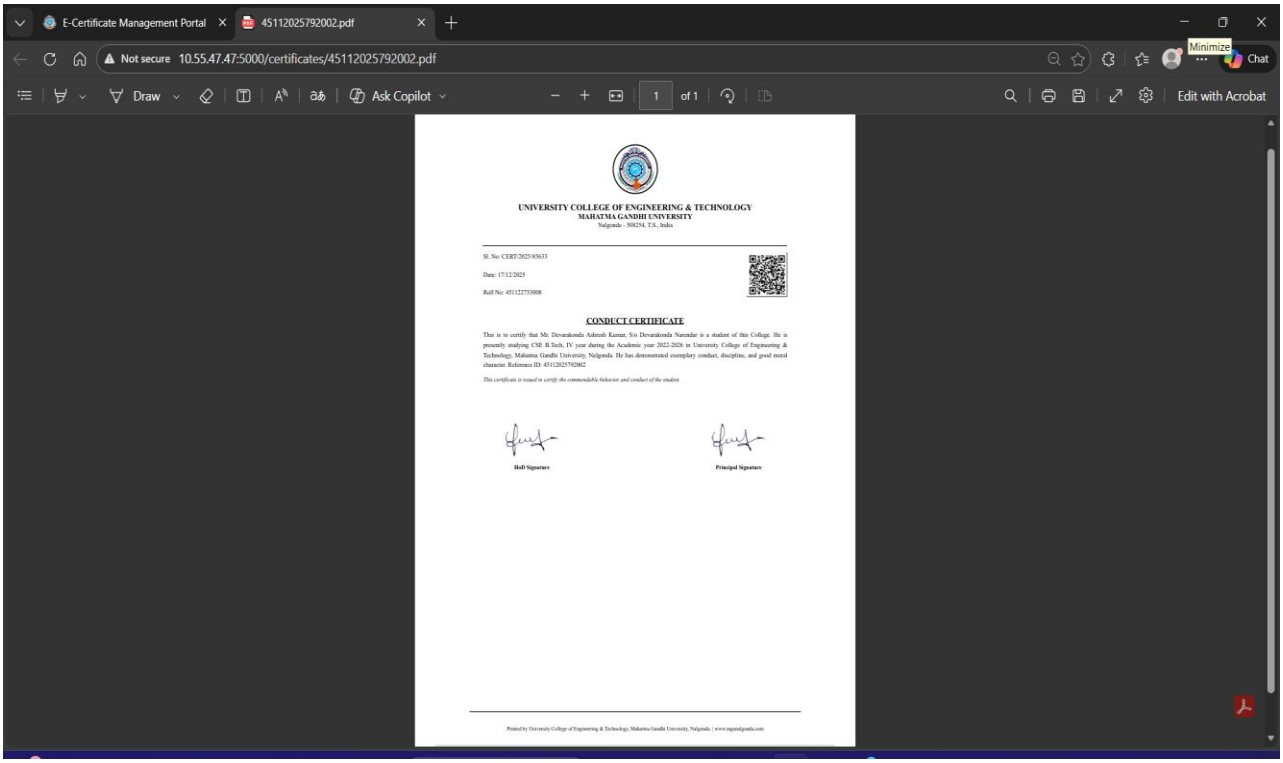

Figure 12.4.6: Print Certificate page

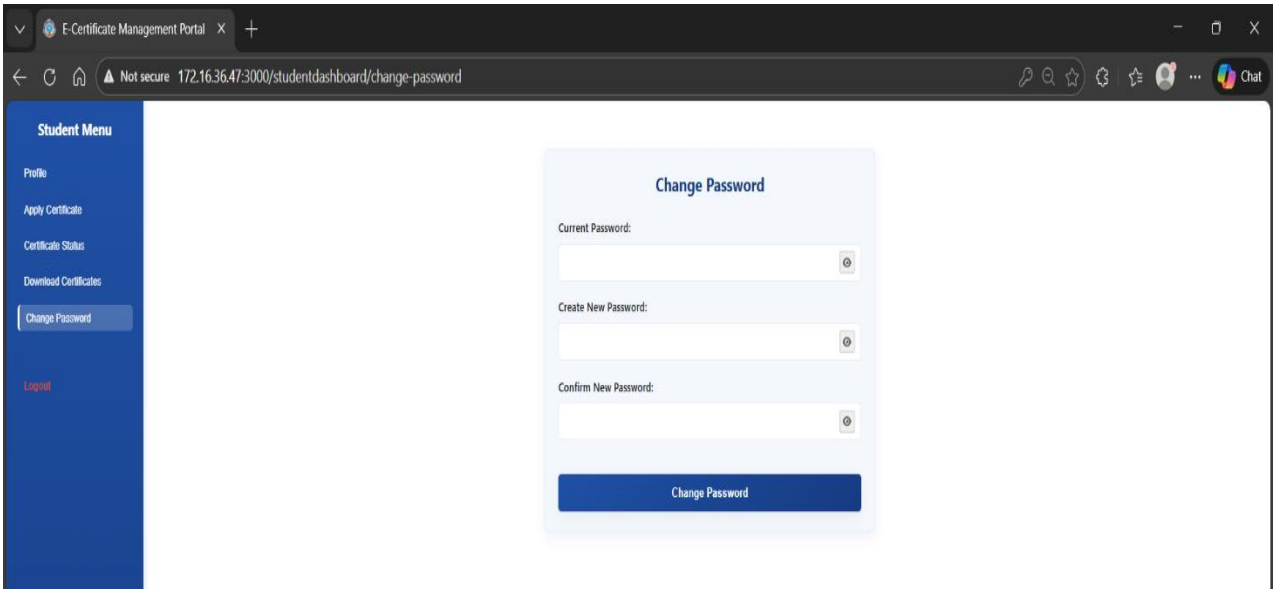

Figure 12.4.7: Change Password page

12.5 Admin Dashboards

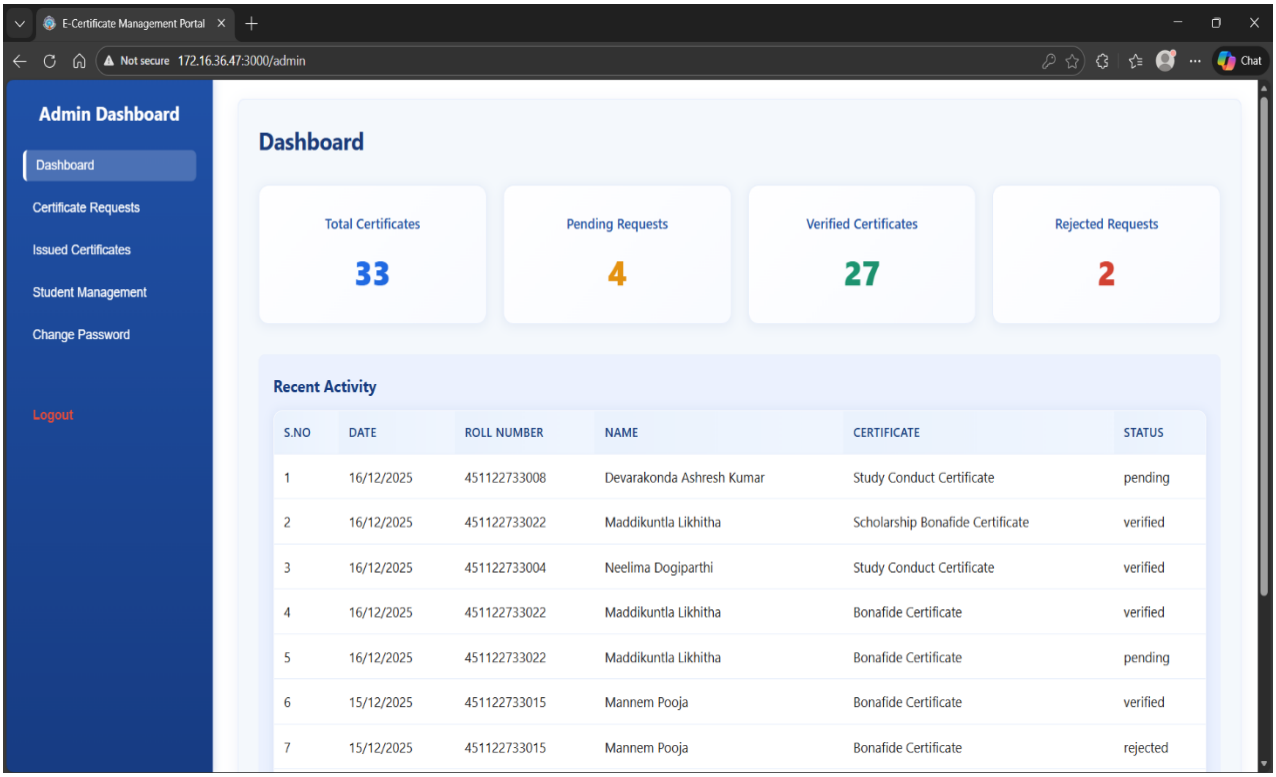

Figure 12.5.1: Admin Dashboard page

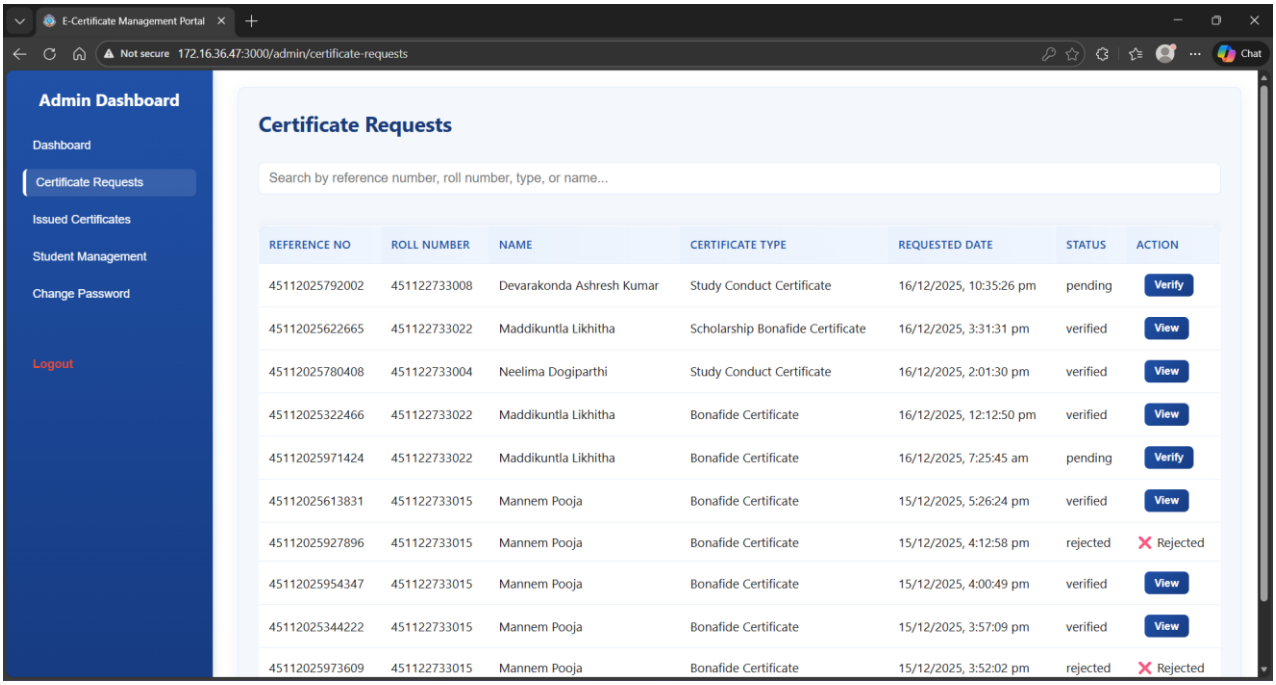

Figure 12.5.2: Certificate Request page

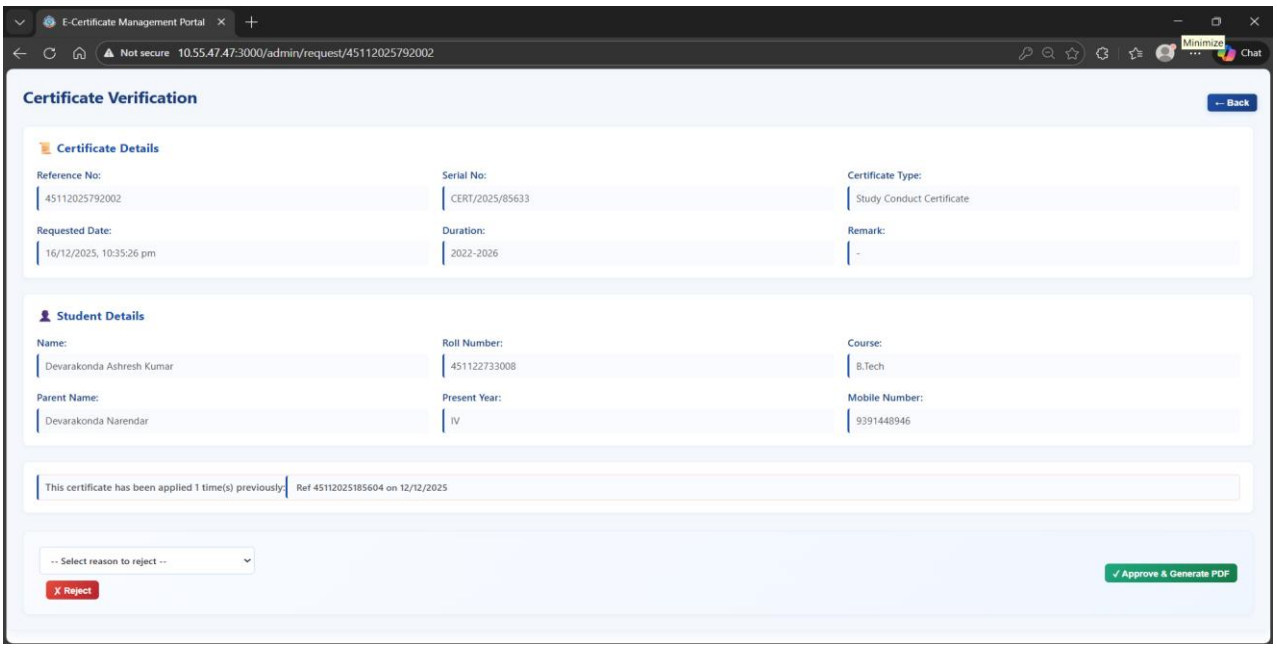

Figure 12.5.3: Certificate Verify Page

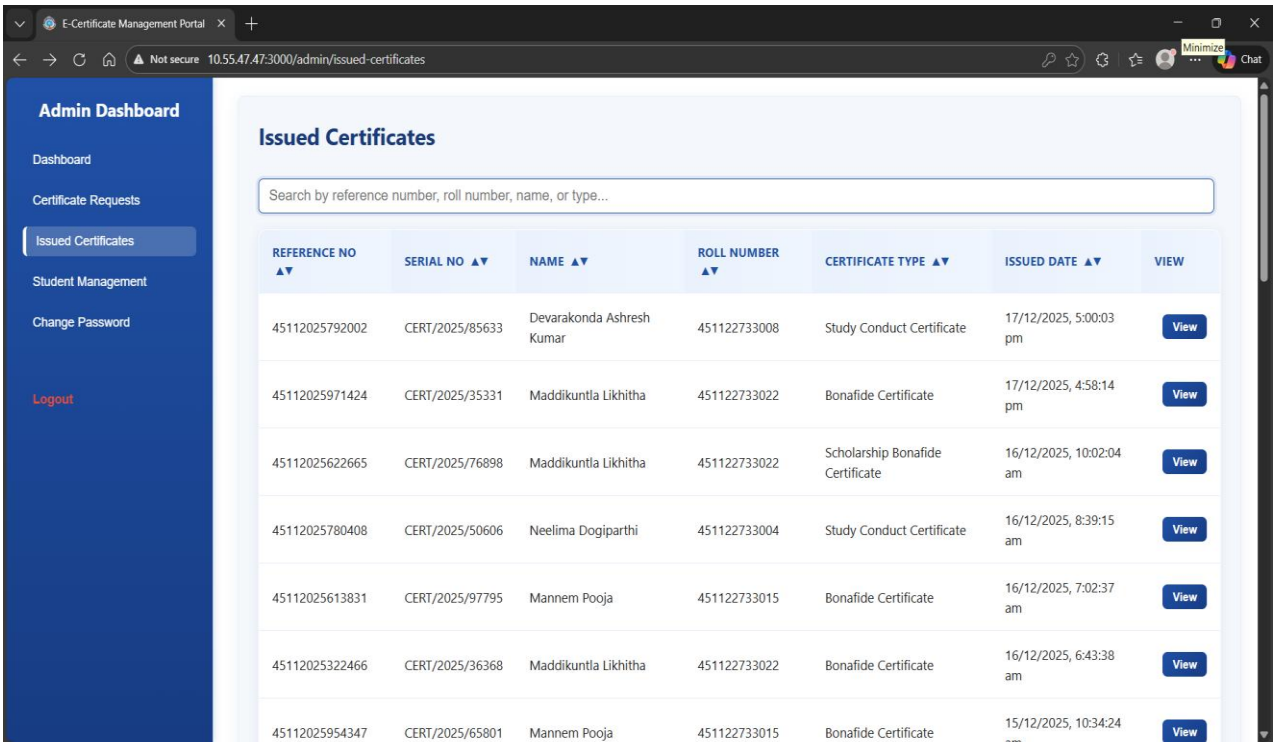

Figure 12.5.4: Issued Certificate Page

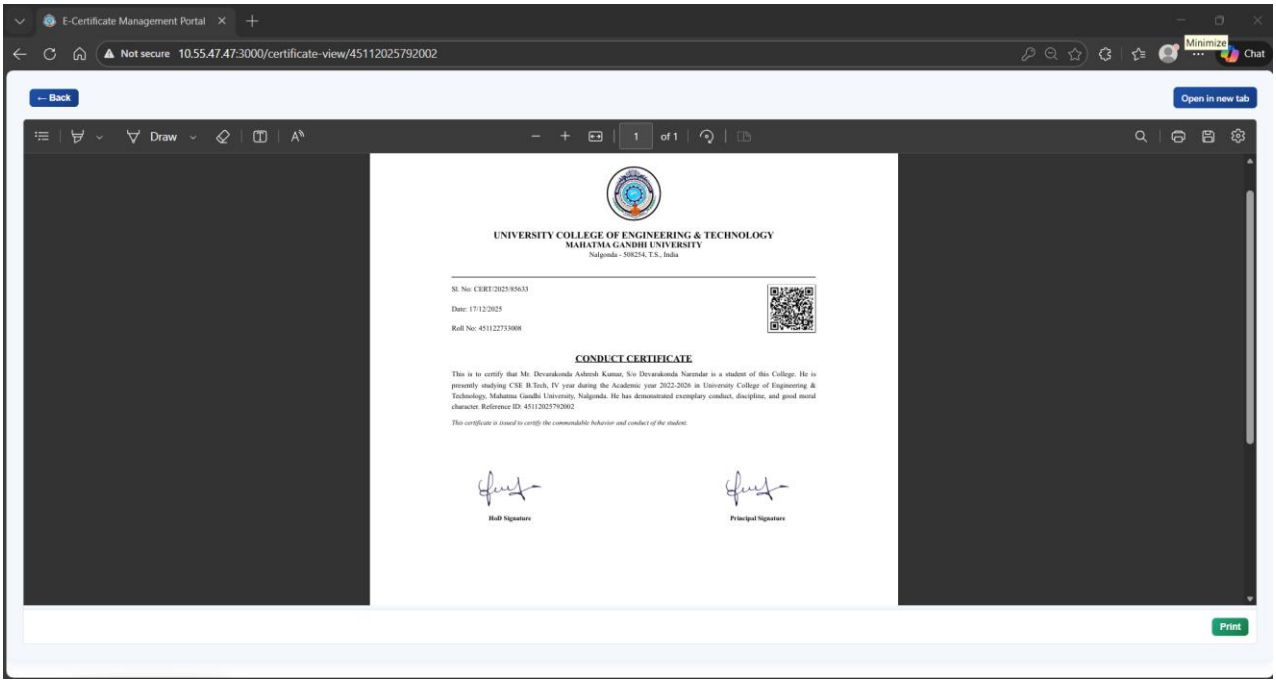

Figure 12.5.5: View & Print Certificate page

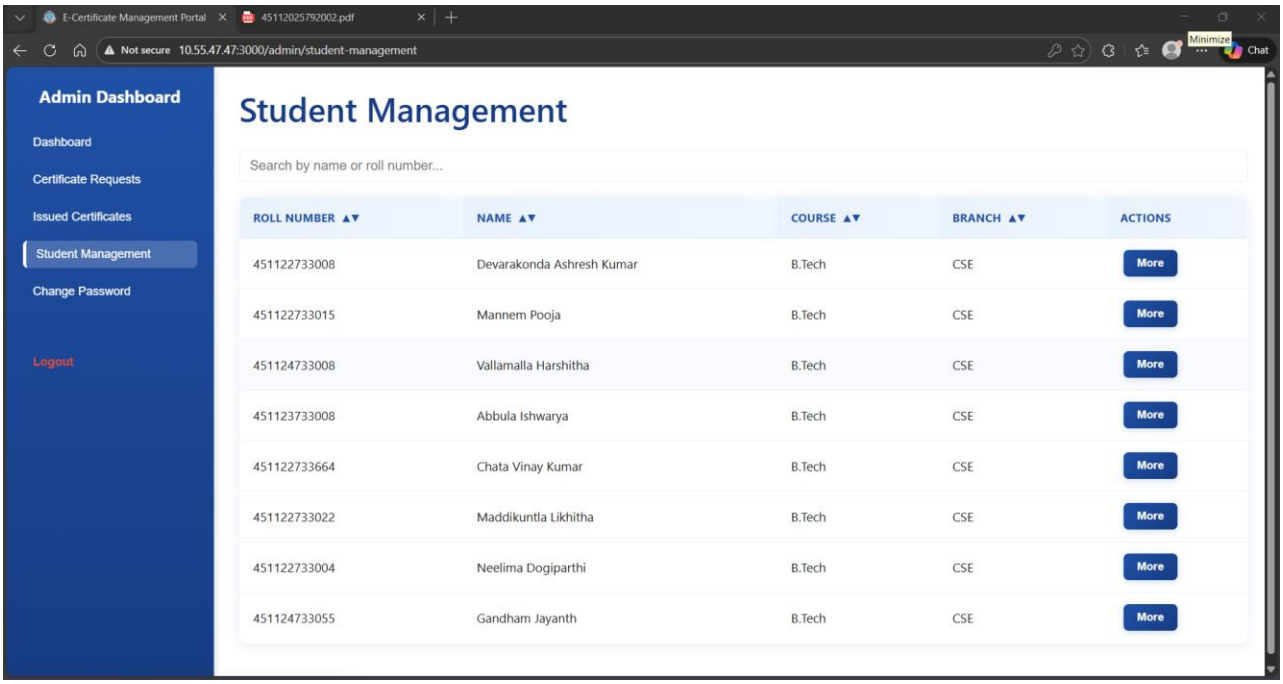

Figure 12.5.6: Student Management page

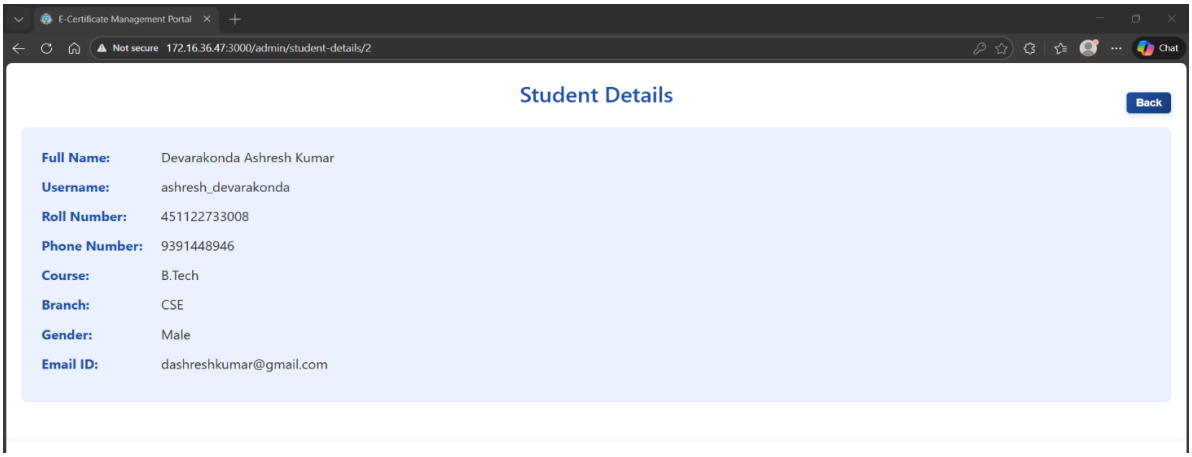

Figure 12.5.7: Student Details page

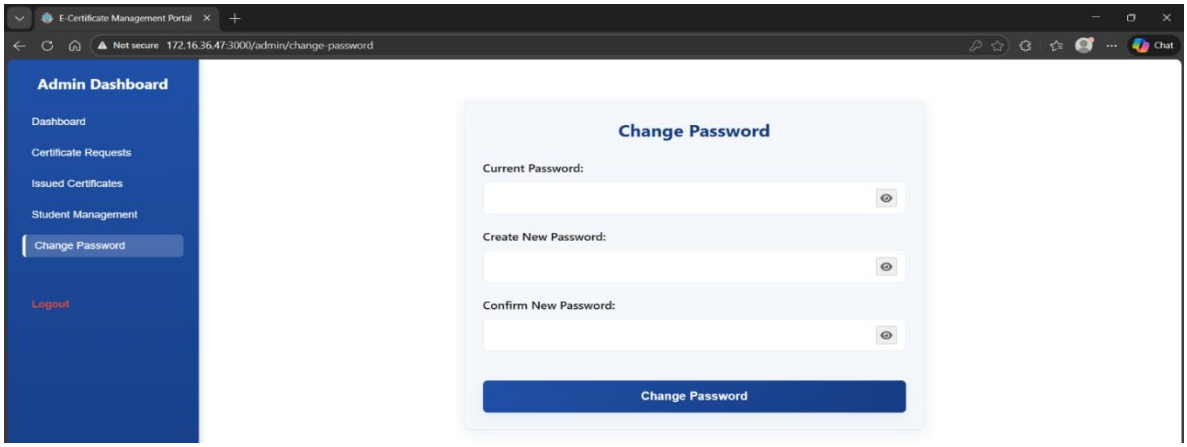

Figure 12.5.8: Change Password page

12.6 Certificate Verification Page

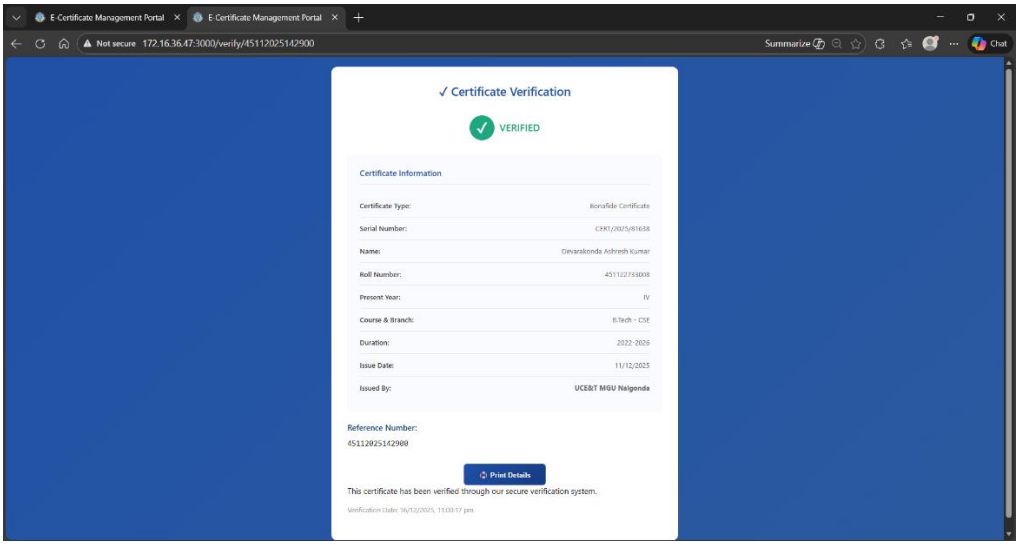

Figure 12.6 Certificate Verification page

12.7 Certificates

UNIVERSITY COLLEGE OF ENGINEERING & TECHNOLOGY
MAHATMA GANDHI UNIVERSITY
Nalgonda - 508254, T.S., India

Sl. No: CERT/2025/41916

Date: 17/12/2025

Roll No: 451122733008

BONAFIDE CERTIFICATE

This is to certify that Mr. Devarakonda Ashresh Kumar, S/o Devarakonda Narendar is a bonafide student of this College. He is presently studying CSE B.Tech, IV year during the Academic year 2022-2026 in University College of Engineering & Technology, Mahatma Gandhi University, Nalgonda. He has maintained satisfactory attendance and academic progress.

This certificate is issued for official use and verification of student enrollment status.

HoD Signature

Principal Signature

Printed by University College of Engineering & Technology, Mahatma Gandhi University, Nalgonda. | www.mgunalgonda.com

Figure 12.7.1: Bonafide Certificate

UNIVERSITY COLLEGE OF ENGINEERING & TECHNOLOGY
MAHATMA GANDHI UNIVERSITY
Nalgonda - 508254, T.S., India

Sl. No: CERT/2025/85386

Date: 17/12/2025

Roll No: 451122733008

CONDUCT CERTIFICATE

This is to certify that Mr. Devarakonda Ashresh Kumar, S/o Devarakonda Narendar is a student of this College. He is presently studying CSE B.Tech, IV year during the Academic year 2022-2026 in University College of Engineering & Technology, Mahatma Gandhi University, Nalgonda. He has demonstrated exemplary conduct, discipline, and good moral character. Reference ID: 45112025356765

This certificate is issued to certify the commendable behavior and conduct of the student.

HoD Signature

Principal Signature

Printed by University College of Engineering & Technology, Mahatma Gandhi University, Nalgonda. | www.mgunalgonda.com

Figure 12.7.2: Conduct Certificate

UNIVERSITY COLLEGE OF ENGINEERING & TECHNOLOGY
MAHATMA GANDHI UNIVERSITY
Nalgonda - 508254, T.S., India

Sl. No: CERT/2025/30874

Date: 17/12/2025

Roll No: 451122733008

BONAFIDE CERTIFICATE

This is to certify that Mr. Devarakonda Ashresh Kumar, S/o Devarakonda Narendar is a bonafide student of this College. He is presently studying CSE B.Tech, IV year during the Academic year 2022-2026 in University College of Engineering & Technology, Mahatma Gandhi University, Nalgonda. Reference ID: 45112025403305

This certificate is issued for the purpose of submitting to scholarship.

Candidate Signature **HoD Signature** **Principal Signature**

Printed by University College of Engineering & Technology, Mahatma Gandhi University, Nalgonda. | www.mgunalgonda.com

Figure 12.7.3: Scholarship Bonafide Certificate

12.8 Performance Metrics

Benchmarked performance shows that common operations (login, request submission, list retrieval) complete within the targeted response time under typical loads. Certificate generation, including QR and PDF creation, completes within a few seconds for single requests on standard hardware.

These results indicate that the system is suitable for deployment in an institutional environment where hundreds or thousands of certificates may be generated each year.

12.9 Security Audit Summary

A basic security review confirms that passwords are not stored in plaintext and that user input to database queries is parameterized. OTP values are not displayed in logs, and verification endpoints do not leak sensitive internal information. While advanced penetration testing and formal audits are beyond the project scope, the current implementation aligns with common web application security best practices.

13. CONCLUSION & FUTURE SCOPE

13.1 Conclusion and Achievements

The e-Certificate Management System successfully automates the end-to-end lifecycle of academic certificates, including request submission, administrative approval, certificate generation, and external verification. The project meets its objectives by providing a secure, user-friendly web interface for students and administrators and by reducing manual workload and processing time.

By integrating QR code–based verification and a centralized database, the system enhances trust in academic documents and makes it easier for external stakeholders to validate credentials. The full-stack architecture demonstrates practical application of concepts studied in web technologies, databases, and software engineering.

13.2 Limitations

The current implementation is designed for deployment within a single institution and assumes a reliable local server and network. It does not yet include advanced features such as blockchain storage, multi-institution federation, or mobile apps. Automated scaling, high-availability clusters, and third-party LMS integration are also outside the current scope.

13.3 Future Enhancements

Potential future enhancements include:

- Blockchain-based verification for higher tamper resistance.
- Development of native Android/iOS applications for students and administrators.
- Support for multi-institution deployments with centralized verification across universities.
- Integration with existing ERP or LMS systems to fetch student data and push issuance records automatically.
- Advanced analytics dashboards showing certificate issuance trends and verification statistics.